Supplementary Material for "ArtiAtlas: A Holistic Ecosystem for Diverse 3D Articulation Modeling"

JUNZHE LU, Tsinghua University, China and ByteDance Seed, China
JING LIN*, S-Lab for Advanced Intelligence, Nanyang Technological University, Singapore and ByteDance Seed, China
ZIANG CAO, S-Lab for Advanced Intelligence, Nanyang Technological University, Singapore
HAOQIAN WANG†, Tsinghua University, China
ZIWEI LIU, S-Lab for Advanced Intelligence, Nanyang Technological University, Singapore
JIANFENG ZHANG†, ByteDance Seed, China

This appendix accompanies the main paper and expands the dataset, evaluation, and model details that did not fit in its page budget. References to the main paper are marked as such; every other number refers to this document.

A Appendix Overview

This appendix expands the dataset, evaluation, and model details that did not fit into the page budget of the main paper. Each appendix is self-contained but cross-references the section it serves.

- **Data Collection and Filtering** (Appendix B). The three intake streams that feed ArtiAtlas: the multi-source raw-asset stream (Appendix B.1) and the public PartNeXt / PartVerse-XL pools (Appendix B.2). Reports the per-stage retention statistics that underlie the asset counts cited in Section 3.
- **Manual Part Consolidation and Curation** (Appendix C). The Blender-based annotation interface, consolidation criteria, and reject-vs-merge decisions used to convert raw mesh decompositions into the motion-aware semantic part manifests consumed downstream.
- **Articulation Annotation Pipeline** (Appendix D). The three stage-level designs of ArtiPipe—kinematic structure (Appendix D.1), joint axis with its prototype catalogue and selection (Appendix D.2, failure modes in Appendix D.2), and motion range with differentiable render-and-fit (Appendix D.3)—plus the human review tool and per-asset annotation efficiency (Appendix D.4). Closes with a random-sample comparison against PhysXNet (Appendix D.5, Figure 12).
- **Final Dataset Statistics and Release** (Appendix E) reports post-review source composition, controlled-vocabulary classification, counts for all 173 level-2 categories, and the released resources.
- **Implementation Details** (Appendix F). Full architecture, loss, and optimisation choices for Artisan (Appendix F.1); the method-specific adaptations and reproduction notes for every comparison method evaluated in Section 6 (Appendix F.2); and the connectivity-based label refinement (Appendix F.3) used to derive the † rows.
- **Benchmark Details** (Appendix G). Conditioning-view sampling, definitions and aggregation rules for paired and reference-free metrics, the VLM diagnostic renders, the human study that calibrates the VLM judge, qualitative judge analysis, the test-set gallery, the Lightwheel reference evaluation, and the PartNet-Mobility legacy split that we retain only as an appendix reference.
- **Failure Analysis** (Appendix H). Additional analysis of the remaining gap to part-mesh pipelines, input-dependent failure modes, and qualitative residuals, with references to the Category-Rich, In-the-Wild, and Artisan-focused galleries.
- **VLM Prompting Details** (Appendix I). The static prompt text used by the annotation pipeline and the shared system prompt and per-aspect rubrics used by the benchmark VLM judge.

B Data Collection and Filtering

We constructed the annotation set from three sources: a large multi-source raw-asset stream and two recently released public part-segmented datasets, PartNeXt [Wang et al. 2025a] and PartVerse-XL [Ding et al. 2025]. The raw-asset stream consists of raw 3D assets without curated part-level annotations, whereas the two public datasets already provide manually verified part decompositions and asset-level semantic labels, so the filtering pipelines diverge in how many early stages they need. The stream combines ten public research datasets and assets from a commercial partner; Appendix E reports its post-review composition and category distribution.

B.1 Multi-source raw-asset stream

For the raw-asset stream, the assets are too numerous for manual review and do not come with curated part annotations. We therefore use a VLM-based pre-processing stage, followed by automatic geometry checks and a final balanced sampling step.

Initial candidate pool. We started from a multi-source raw-asset stream containing over one million rendered 3D assets. The pool combines ten publicly available research datasets—ShapeNetCore [Chang et al. 2015], Objaverse-XL [Deitke et al. 2023], ABC [Koch et al. 2019], ModelNet [Wu et al. 2015], 3DCoMPaT200 [Ahmed et al. 2024], HSSD [Khanna et al. 2024], 3D-FUTURE [Fu et al. 2021], Fusion 360 Gallery [Willis et al. 2021], AI2-THOR [Kolve et al. 2017], and FG3D [Liu et al. 2021]—with assets from a commercial partner. Each

*Project lead.
†Corresponding authors.

Authors' Contact Information: Junzhe Lu, Tsinghua University, Shenzhen, China and ByteDance Seed, Beijing, China, junzhelu.thu@gmail.com; Jing Lin, S-Lab for Advanced Intelligence, Nanyang Technological University, Singapore, Singapore and ByteDance Seed, Beijing, China, jinglin.stu@gmail.com; Ziang Cao, S-Lab for Advanced Intelligence, Nanyang Technological University, Singapore, Singapore, ziang001@e.ntu.edu.sg; Haoqian Wang, Tsinghua University, Shenzhen, China, wanghaoqian@tsinghua.edu.cn; Ziwei Liu, S-Lab for Advanced Intelligence, Nanyang Technological University, Singapore, Singapore, zwliu.hust@gmail.com; Jianfeng Zhang, ByteDance Seed, Beijing, China, jf.zhang958@outlook.com.

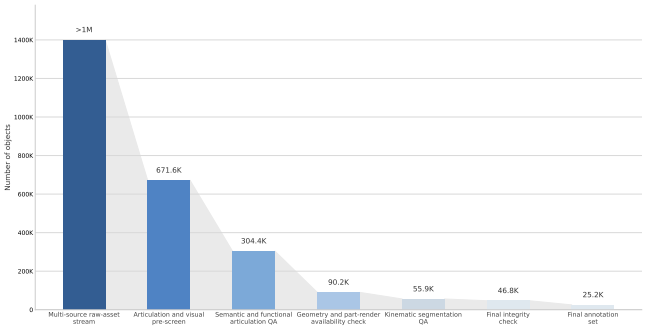


Fig. 1. Filtering funnel used to construct the raw-asset-stream annotation pool. The number above each bar shows how many assets remain after the corresponding stage; similar checks are grouped for readability.

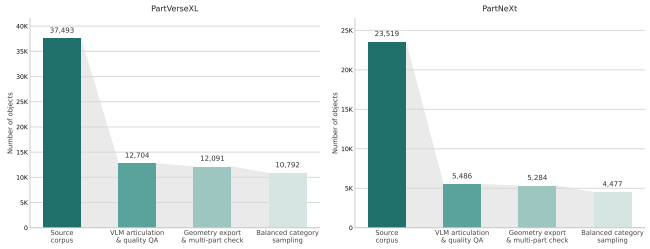


Fig. 2. Filtering funnel for the two public part-structured sources, PartNeXt and PartVerse-XL. Both pipelines are shorter than the raw-asset-stream pipeline because the source assets already ship with curated part decompositions and asset-level semantic labels. The number above each bar shows how many assets remain after the corresponding stage.

candidate has a unique asset identifier and at least one rendered image used for automated visual screening. The initial part structure is not manually annotated: assets are decomposed into candidate parts by an algorithmic mesh-connectivity procedure. These connected components are treated as candidate rigid parts and later filtered for semantic validity, geometry quality, and kinematic separability.

Visual and semantic filtering. We first use a VLM to score each rendered asset for whether it visually contains articulated parts and whether the mesh appears usable. We keep objects predicted to contain articulation with high confidence and acceptable visual quality, producing 671.6k candidates. We then run additional VLM-based semantic and functional QA to reject static false positives, loose multi-object scenes, exploded assemblies, illogically detached parts, and severe visual defects. We also assign a three-level taxonomy during this stage. After this merged semantic QA stage, 304.4k objects remain.

Geometry and render-availability filtering. We next check whether the underlying part geometry is usable for physical annotation. Assets with excessive open or thin-shell geometry are removed using a part-level geometric test that tolerates minor defective parts but rejects assets whose bad part surface area exceeds a threshold. We also require the part-segmentation and normal renders needed for later kinematic QA to be available. These checks reduce the pool from 304.4k to 90.2k objects.

Kinematic segmentation and integrity QA.. The remaining objects are screened using part-segmentation and normal-map renderings. We use VLM-based QA to reject assets where moving components are under-segmented or fused with the body, as well as static shells and functionally incomplete objects. This stage produces 55.9k candidates. A final VLM integrity check then verifies semantic plausibility, expected part counts, and coarse physical validity, rejecting cases with missing seams, floating parts, missing structure, or scattered geometry. This leaves 46.8k valid articulated candidates.

Final sampling. Before annotation, we remove objects with at most two visible parts, leaving 43.1k candidates. To avoid domination by frequent level-2 categories, we retain all objects from rare categories and sample frequent categories with a square-root allocation rule. This produces the final pool of 25.2k objects sent to the annotation pipeline.

B.2 PartNeXt and PartVerse-XL

PartNeXt provides 23,519 annotated rows distributed over multiple Hugging Face Arrow shards, of which 23,229 have an object-level GLB available at retrieval time, and PartVerse-XL provides 37,493 textured part bundles, each consisting of per-part GLBs together with a per-asset annotation JSON. Unlike the raw-asset stream, both sources already ship with curated part-level decompositions and human-verified semantic labels, and PartVerse-XL further provides per-part textured GLBs. Because these two sources have already passed the manual semantic-decomposition step that the raw-asset stream's mesh-connectivity stage approximates, the early visual articulation pre-screen, semantic functional QA, and geometry-availability checks used in Section B.1 become unnecessary. The filtering pipeline therefore collapses to three substantive stages on top of the source corpus. We use the same Blender rendering protocol as for the raw-asset stream to obtain four orbital RGB views and a part-color mask per asset. The corresponding funnel is summarized in Figure 2.

VLM articulation and quality QA.. For each sample, we render four textured RGB views and four aligned part-color views using a fixed orbital camera schedule: $(45°, 30°)$, $(135°, 30°)$, $(225°, 30°)$, and $(315°, 30°)$, all at a resolution of 1280×720. The four RGB renders are packed into a single 2×2 grid, and the four part-color renders are packed into a second 2×2 grid with identical viewpoint ordering. These two aligned PNG grids are the only image inputs to the VLM for both PartNeXt and PartVerse-XL; representative examples are shown in Figure 3. The part-color rendering assigns each part a flat, distinct color so that likely moving components and static support structures can be judged reliably from the segmentation alone.

We pair the two image grids with a single fixed instruction for articulated-object filtering; the full prompt is reproduced as Prompt 1 in Section I. The prompt jointly predicts four gating outputs and a three-level taxonomy. The articulation gate is strict: an asset counts as articulated only if the textured appearance supports relative motion between rigid parts through a revolute, prismatic, or screw joint. Doll-like rigs, deformation-driven or soft-body motion, and

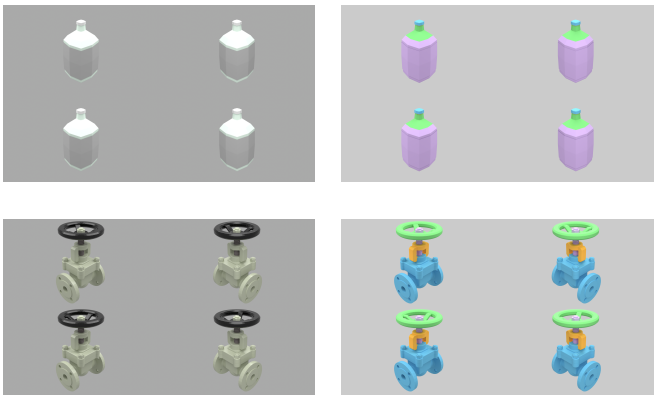


Fig. 3. Example VLM inputs for the articulated-object filtering stage. In each row, the left image is the textured RGB four-view collage and the right image is the part-colored four-view collage.

plug-in/snap-fit block assemblies are explicitly excluded. The remaining gates test whether the object is recognizable as a complete artefact, whether the part-color rendering under-segments moving components, and whether the mesh is structurally complete and free of severe artifacts. The same instruction also assigns a mutually exclusive level-1 super-category together with level-2 and level-3 `snake_case` names, and requires the VLM to return only a JSON object containing the sample identifier, all gate decisions, the taxonomy, and a short rationale. An asset survives this stage only when all four gates pass. The articulation gate is the most aggressive, removing 54.07% of PartVerse-XL and 72.76% of PartNeXt. Failures attributable only to the remaining integrity gates are much less common: a further 12.0% of PartVerse-XL and 3.3% of PartNeXt are dropped at this stage. The VLM stage retains 12,704 PartVerse-XL assets and 5,486 PartNeXt assets.

Geometry export and multi-part check. Surviving assets are converted to OBJ format with deduplicated PNG textures, and their part decomposition is re-checked: assets whose decomposition collapses to fewer than two non-degenerate meshes after GLB→OBJ conversion are dropped. This stage removes a further 613 PartVerse-XL assets and 202 PartNeXt assets, leaving 12,091 and 5,284 candidates respectively.

Balanced category sampling. Finally, we apply the same square-root allocation rule used for the raw-asset-stream pipeline. Every asset from rare level-2 categories, defined as those containing fewer than 50 surviving assets in the source, is retained, while frequent categories are sub-sampled proportionally to the square root of their size. The final pools shipped to annotation contain 10,792 PartVerse-XL and 4,477 PartNeXt assets; together with the raw-asset-stream annotation pool, they constitute the full set of articulated-object candidates used in the rest of the paper.

C Manual Part Consolidation and Curation

The source-specific data-collection funnels described above produce candidate assets that are visually articulated and broadly usable, but many are still over-segmented for downstream kinematic fitting.

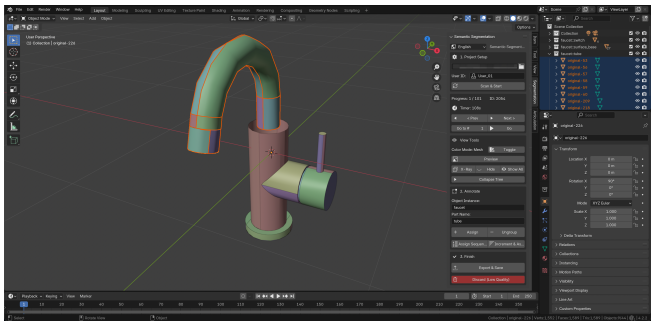


Fig. 4. Blender-based interface for manual part consolidation. Annotators inspect the current decomposition, box-select low-level geometric pieces, merge them into semantic parts, and use a dedicated discard action when an asset remains unsuitable for articulation annotation.

A single rigid component is often split into many low-level mesh fragments, which makes the raw decomposition too fine-grained to serve directly as the semantic part structure used by the articulation annotators. We therefore insert a manual consolidation stage before articulation annotation. Annotators inspect each retained asset in a custom Blender plugin, box-select geometric fragments, merge them into motion-aware semantic parts, rename the merged parts, and discard samples that still cannot support a reliable articulation annotation.

Semantic part manifest. Each curated asset is stored as a *semantic part manifest*: a structured inventory of the object's functional parts together with the OBJ fragments assigned to each part. This representation plays a role similar to a compact object-structure manual: it records how raw mesh pieces are grouped into semantically meaningful rigid units such as a base, cap, lid, handle, or drawer front. Figure 4 illustrates the plugin-based grouping workflow, while the finalized manifest is the direct input to the downstream articulation annotation pipeline. For the running faucet example (a PartNet-Mobility sample used for illustration), the manifest contains a rotating switch, the fixed surface-mounted base, and the swivelling tube. The object lists are abbreviated below.

Example semantic part manifest

```
[
  {
    "id": 0,
    "name": "switch",
    "objs": [
      "textured_objs/original-105.obj",
      "textured_objs/original-133.obj",
      "...",
      "textured_objs/original-185.obj"
    ]
  },
  {
    "id": 1,
    "name": "surface_base",
    "objs": [
      "textured_objs/original-17.obj",
```

```
        "textured_objs/original-173.obj",
        "...",
        "textured_objs/original-99.obj"
      ]
    },
    {
      "id": 2,
      "name": "tube",
      "objs": [
        "textured_objs/original-209.obj",
        "textured_objs/original-218.obj",
        "...",
        "textured_objs/original-60.obj"
      ]
    }
  ]
```


Final pre-annotation filtering. This manual consolidation stage also acts as a final pre-annotation quality gate. Annotators discard assets that passed the earlier automatic filters but remain structurally unusable: missing hinge or sliding interfaces, movable components that are still under-segmented and cannot be separated at the mesh level, meshes with severe quality artefacts, and objects with missing structural parts. Relative to the pools at the end of Section B, this step reduces the raw-asset stream from 25,225 to 11,049 assets, PartNeXt from 4,477 to 1,861, and PartVerse-XL from 10,792 to 1,752. In total, 14,662 curated assets remain after manual consolidation and are forwarded to the subsequent articulation annotation pipeline.

D Articulation Annotation Pipeline

This appendix expands the three components introduced in Section 3.3: kinematic structure prediction, joint-axis estimation, and motion-range estimation. Rather than presenting the system as a sequence of engineering stages, we organize the description around the type of uncertainty each component resolves and the signal that best constrains it. Structure is primarily a commonsense recognition problem with strong object-level priors. Axis estimation is a continuous geometric problem in which the meshes themselves carry most of the information. Motion range combines geometric scale with a physically grounded sign convention that depends on the chosen axis direction.

D.1 Kinematic structure

The first sub-problem is to predict a kinematic tree over the semantic parts and assign a joint type to each non-root part. We formulate this stage as a VLM-based recognition problem. Given the reference render and per-part highlight renders shown in Figure 5, together with the semantic part manifest, the model predicts a parent identifier and a joint type from the set {`revolute`, `prismatic`, `continuous`, `screw`, `fixed`}. The full instruction is reproduced as Prompt 2. The prompt is organized as a compact mechanical knowledge base rather than as a free-form instruction: it includes short definitions of the joint types, a small number of object-level heuristics that resolve recurring ambiguities, and a curated exemplar that anchors the expected output format.

On the running faucet example, the model identifies the surface-mounted base as the root, assigns a revolute joint to the handle switch, and assigns a continuous joint to the spout tube. The resulting typed tree is the kinematic-structure output consumed by the axis stage, with the object lists abbreviated below.

Example kinematic-structure output


```
[
  {
    "id": 0,
    "name": "switch",
    "objs": [
      "textured_objs/original-105.obj",
      "...",
      "textured_objs/original-185.obj"
    ],
    "parent": 1,
    "joint": {
      "type": "revolute"
    },
    "rationale": "The blue lever is attached to the main
     faucet base and rotates to control water flow and
     temperature."
  },
  {
    "id": 1,
    "name": "surface_base",
    "objs": [
      "textured_objs/original-17.obj",
      "...",
      "textured_objs/original-99.obj"
    ],
    "parent": -1,
    "joint": {
      "type": "fixed"
    },
    "rationale": "The orange base is the stationary
     component that mounts directly to the sink surface,
     serving as the root for all other moving parts."
  },
  {
    "id": 2,
    "name": "tube",
    "objs": [
      "textured_objs/original-209.obj",
      "...",
      "textured_objs/original-60.obj"
    ],
    "parent": 1,
    "joint": {
      "type": "continuous"
    },
    "rationale": "The green spout is designed to swivel 360
     degrees or across a wide arc relative to the fixed base
      to direct water flow."
  }
]
```


This formulation is appropriate because the dominant uncertainty at this stage is functional rather than geometric. Deciding whether a part is a door, a drawer, a knob, or a hinged cover often depends more on object-level priors than on precise mesh fitting. On a complete re-annotation of PartNet-Mobility [Xiang et al. 2020] (2,347 assets), this stage reaches 96.9% parent-assignment accuracy and 96.0% joint-type accuracy. The remaining errors are concentrated

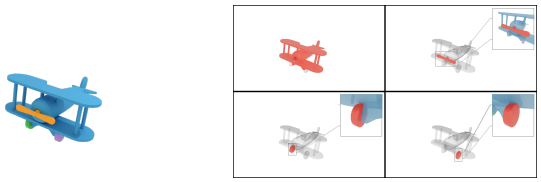


Fig. 5. Renders consumed by the kinematic-structure stage. Left: a normalized orbital RGB render of the full asset that supplies global context. Right: per-part highlight renders, in which the queried part remains opaque while every other part is rendered as a semi-transparent ghost. The VLM uses these to reason about each part individually without losing visibility into how the part sits in the rest of the object.

in assets whose segmentation is already imperfect, or in borderline distinctions such as screw versus revolute caps.

D.2 Joint axis: prototype catalogue and selection

The second sub-problem is to recover a continuous 3D axis, specified by an origin and a direction, for each non-fixed joint. Pilot experiments confirmed that asking a VLM to emit numeric axis parameters directly is brittle, both because the model must invent six floating-point numbers in a single decoding pass and because current VLMs still exhibit limited 3D spatial grounding. The mesh, in contrast, typically exposes a usable geometric cue for the correct axis. We organize these cues into a catalogue of named *prototypes*. Each prototype is a small, hand-engineered geometric program with a fixed input–output contract: it consumes the parent and child meshes (and, when relevant, a precomputed contact region between them), isolates the cue it is built for, and returns zero or more candidate axes together with provisional limits.

The catalogue partitions into two semantic families that mirror the two cue types observed across articulated objects. *Part-intrinsic* prototypes recover the axis from a single part's own geometry, exploiting symmetries of either the moving part or its parent frame in isolation, as in a wheel whose rotation axis is the geometric axis of its disc, or a clock pointer whose rotation axis is the geometric axis of the surrounding bezel. *Contact-region* prototypes recover the axis from the spatial relationship between the moving part and its parent, fitting a primitive such as a ring, plane, line, or polygon to the points where the two meshes meet, as in a closed cabinet door whose hinge lies on one edge of the rectangular contact between door and frame. Table 1 lists the prototypes we expose to the VLM, together with a one-line implementation summary, representative objects on which each is the natural choice, and the joint types it is registered for.

Axis prediction proceeds in two stages. A proposal VLM first reads the per-part renders together with the typed kinematic tree and selects a small ordered subset of prototypes for each joint, returning at most three names per joint with a short rationale, as specified in Prompt 3. The selected prototypes are then executed geometrically, and near-duplicate outputs are clustered into a compact candidate set. A second VLM is then shown the rendered candidate axes (Figure 6), the predicted joint type, and a short block of physical priors stating that door hinges oppose handles, drawer slide axes are perpendicular to the cabinet front, wheel axes pass through the visible centre, and screw axes follow the threaded cylinder. Conditioned on these priors and the candidate set, it returns the single best candidate under Prompt 4.

For the faucet, the selector chooses the contact-ring axis through the handle pivot and the vertical swivel axis through the spout neck. At this point the range values are provisional placeholders. The downstream stage uses the selected origin and direction values as fixed axis inputs.

Example selected-axis output


```
[
  {
    "id": 0,
    "parent": 1,
    "joint": {
      "type": "revolute",
      "range": [0, 90],
      "axis": {
        "origin": [0.2644, -0.3026, -0.1357],
        "direction": [1, 0, 0]
      }
    },
    "selection_id": 0,
    "quality": "perfect"
  },
  {
    "id": 1,
    "parent": -1,
    "joint": {
      "type": "fixed",
      "range": [0, 0],
      "axis": {
        "origin": [0, 0, 0],
        "direction": [0, 0, 1]
      }
    },
    "selection_id": 0,
    "quality": "perfect"
  },
  {
    "id": 2,
    "parent": 1,
    "joint": {
      "type": "continuous",
      "range": [0, 0],
      "axis": {
        "origin": [-0.0923, 0.1886, -0.1356],
        "direction": [0, -1, 0]
      }
    },
    "selection_id": 0,
    "quality": "perfect"
  }
]
```


Axis accuracy and failure modes. We adopt the per-joint success criterion of ArtiBench (orientation error and origin error against ground truth, with the latter normalised by the child mesh's bounding-box diagonal). We tighten the angular tolerance to 3° while keeping

Prototype	Brief implementation	Common applications	Type
Part-intrinsic prototypes (cue: a single part's geometry, child or parent)			
`child_circle`	OBB rim fit on the child mesh; circle normal on the projected convex hull is the axis.	wheels, rollers, dial knobs	C, S
`child_obb`	All three child-OBB principal directions emitted as slide-axis candidates.	sliding blades, inserts without exposed contact	P
`child_general`	Single child-OBB/PCA axis through the child centroid, chosen by an adaptive shape rule.	irregular knobs and shafts; default fallback	C, S
`parent_circle`	Same rim fit, run on the parent mesh when it supplies the circular frame.	dial pointers, gauge needles	C, S
Contact-region prototypes (cue: child–parent interface)			
`contact_ring`	Most-circular projection in the contact SVD frame; axis = ring/-cylinder normal.	hinge barrels, faucet pivots, threaded caps	R, P, C, S
`contact_disc`	Plane + circle fit on face-to-face contact patches; axis = plane normal.	flat flanges, twist-off lids, disc dials	R, P, C, S
`contact_seam`	Line fit on the near-contact seam between child and parent.	exposed door and hinge seams	R
`contact_rectangle`	Four axes along a rectangular contact footprint.	oven doors, cabinet sliders	R
`contact_pointer`	Parent-first contact rectangle; emits two normals at the short-edge midpoints, pulled inward by 10% of the long edge.	dial pointers, gauge needles	R, C
`contact_hole`	Maximum empty circle within a surrounding contact cluster.	scissors, pliers, cross-pivots	R
`contact_button`	Contact OBB; minor in-plane axis through the contact centre as the rocker pivot.	rocker switches, wall plates	R
`contact_rail`	Rail OBB on the contact; slide axis chosen by child/parent span ratio, span set by SDF clearance.	window panels, framed sliders	P
`contact_normal`	Adaptive contact-OBB axis anchored at the contact centroid; generic fallback.	generic knobs and handles; fallback	R, C, S

Table 1. Catalogue of named prototypes exposed to the VLM as a manual. *Part-intrinsic* prototypes (top) recover the axis from one part in isolation, either the moving part or its parent frame, while *contact-region* prototypes (bottom) recover it from the interface between the two. The same prototype is reused across joint types whenever the underlying primitive is the same. R, P, C, and S in the rightmost column denote revolute, prismatic, continuous, and screw joints.

the origin tolerance at the benchmark's 3% of the bounding-box diagonal. On the same re-annotation of PartNet-Mobility, our pipeline reaches 80.5% axis accuracy. Replacing the selector with an oracle that inspects the emitted candidates raises this to 86.1%, which isolates selector ambiguity as one measurable source of error. The qualitative failures in Figure 8 focus on limitations of the geometric catalogue itself: simplified geometry can remove the physical hinge cue, nearby surfaces can create a stronger but spurious contact primitive, and unsupported joint families such as ball joints cannot be represented by a single one-degree-of-freedom axis.

D.3 Motion range

The final sub-problem is to estimate the admissible motion range for non-fixed joints with finite stops. Continuous joints are carried through with a placeholder range, since they have no bounded mechanical limit. We use a visual-generation primary path together with a VLM-driven fallback. The reason for this hybrid design is practical: range combines both scale and sign. Pure language-based estimates are often too weak for distances such as drawer extension, while even for revolute joints the sign of a plausible angle depends on the chosen axis direction and is easy to invert without an explicit geometric cue. In our experiments, VLMs often produced qualitatively plausible answers but were not sufficiently reliable on these two points for annotation.

A natural alternative to learned reasoning is to fix the sign of motion directly from geometry: rotate or translate the moving part by a small step in each direction and pick the one whose configuration is free of self-penetration with the parent. We considered this route early on and observed two systematic failure modes that prevented us from relying on it. The first is semantic: many controls admit no purely physical sign, and the convention is fixed by function rather than by clearance. A representative case is the lever faucet handle in Figure 9, whose lever can swing freely on either side of the rest pose, and whether "open" corresponds to lifting the lever up or pushing it down is a design convention rather than a geometric fact. The second is asset-driven: human artists routinely simplify

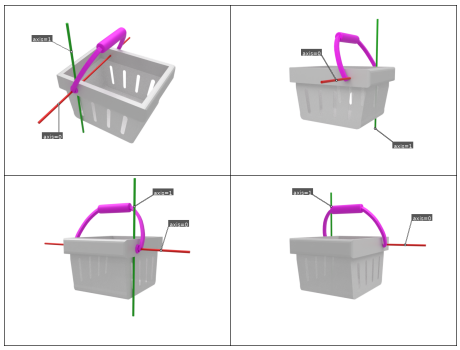


Fig. 6. Four-view candidate-axis grid consumed by the selector VLM. All proposed axes for a single joint are overlaid on each view, drawn from a fixed eight-colour palette and labelled by index, so the model can refer to a specific candidate unambiguously across the four orbital viewpoints.

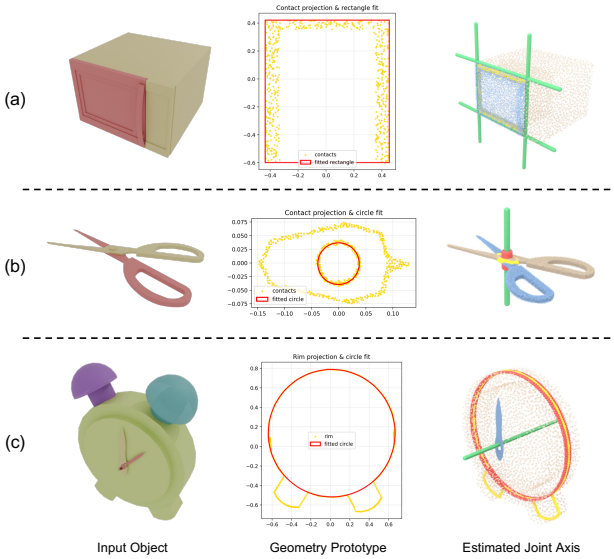


Fig. 7. Three successful prototype invocations from our catalogue. Each row presents the input mesh (left), the intermediate geometric fit recovered by the prototype (middle), and the resulting axis or candidate axes overlaid on the assembly (right). (a) A closed cabinet door, recovered by `contact_rectangle`: an oriented rectangle is fit to the door–frame contact and its four perimeter edges become candidate hinge axes. (b) Scissors, recovered by `contact_hole`: the prototype isolates the empty pivot circle within the contact polygon between the two blades. (c) An alarm-clock pointer, recovered by `parent_circle`: a circle is fit to the parent bezel, and its normal becomes the rotation axis.

modelled geometry, for example embedding rocker switches and push buttons flush with their housings instead of carving a recessed groove, so an SDF-based clearance check sees both directions as equally constrained on assets that physically admit only one. Because both failure modes are common in PartNet-Mobility-scale assets, we leave the sign decision in the range stage to visual generation, which carries object-level priors implicitly.

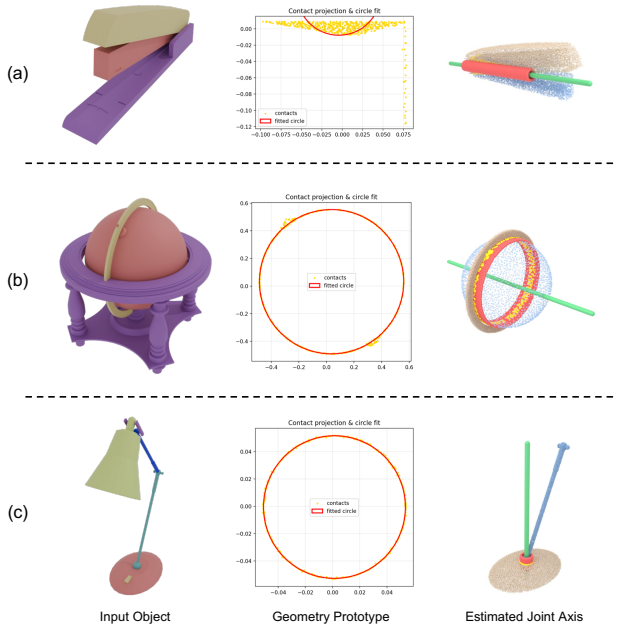


Fig. 8. Three representative failure modes of the axis stage, presented in the same input–prototype–axis layout as Figure 7. (a) Over-simplified modelled geometry: the stapler's outer shell and staple cartridge are modelled without an explicit hinge pin, so the true rotation axis has no surface counterpart in the asset. The detected contact instead clusters on a single side wall, which carries no usable axis cue. (b) Spurious dominant contact: on a desktop globe, the shell sits close enough to the outer ring that a near-perfect circle of contacts is detected on the shell–ring gap, in addition to the small ring at the true pivot. The spurious contact dominates the heuristic, which commits to its axis rather than to the true rotation axis. (c) Out-of-catalogue joint family: the lamp's swing arm meets the base through a ball joint, which our pipeline does not currently model. Forcing `contact_ring` on the spherical contact returns a single vertical axis that cannot represent free rotation about the ball.

Beyond VLMs, current models with useful world knowledge also include video-generation and image-generation or editing models, which suggests an analysis-by-synthesis approach: synthesize a mechanically plausible target state and then optimize the scalar joint value so that a rendered articulation matches the synthesized observation. We initially explored video generation as a source of supervisory trajectories, for example by asking the model to generate a closing motion from an image of an already open drawer. In practice, however, even a recent closed-source video-generation model such as Veo3.1 [Google DeepMind 2025b] was not stable enough for this use case: the camera would often drift despite instructions to remain fixed, and rigid parts could undergo non-rigid distortions. Because stable camera geometry is critical for render-and-fit, and because annotation cost also matters at dataset scale, we instead use an image-editing model, Nano Banana Pro [Google DeepMind 2025a], as the main implementation of the visual path. This choice preserves the reference viewpoint and provides a single target state, which is sufficient for the one-dimensional optimization used below.

Visual generation, occlusion handling, and critic-gated retries. For each non-fixed joint, a planning VLM first receives the reference render, the per-part highlight renders, the semantic part manifest, and the typed kinematic tree. It decides whether visual estimation is appropriate under Prompt 5 and, if so, produces a natural-language edit instruction describing the target configuration near the mechanical limit. The planner additionally returns a list of *occluding part identifiers* (`remove_part_ids`) that block the target part in the reference frame: a typical case is an oven tray hidden behind a closed door, or a rack of internal shelves whose motion is invisible until the cabinet front is hidden. Before editing, we re-render the source frame with these parts removed using the same camera, so that the image-editing model is shown a view in which the target part is fully visible. We use Nano Banana Pro as the editing model, conditioned on this source frame and the planner-authored instruction.

Across the full ArtiAtlas production run, the planner routes approximately 14% of joints to the visual-generation path, concentrating this more costly path on large-travel parts whose motion sign and absolute travel are difficult to estimate directly. Small or repetitive controls, such as knobs and buttons, are instead routed directly to the signed VLM fallback.

The faucet plan routes the handle switch through the visual path and skips the continuous spout, whose unbounded swivel is represented symbolically. No occluder removal is needed in this example, and the edit instruction is shortened in the listing.

Example range-estimation plan

```
[
  {
    "part_id": 0,
    "part_name": "switch",
    "parent_id": 1,
    "joint_type": "revolute",
    "pipeline": "visual",
    "skip_reason": null,
    "current_state": "upright and closed",
    "target_state": "fully tilted forward and open",
    "image_edit_prompt": "Tilt the BLUE faucet handle
      forward 90 degrees to its fully open position.",
    "remove_part_ids": []
  },
  {
    "part_id": 2,
    "part_name": "tube",
    "parent_id": 1,
    "joint_type": "continuous",
    "pipeline": "skip",
    "skip_reason": "continuous",
    "current_state": null,
    "target_state": null,
    "image_edit_prompt": null,
    "remove_part_ids": []
  }
]
```

Synthesized target frames are not consumed directly. They are first inspected by a permissive critic VLM under Prompt 6, biased toward acceptance: it focuses only on the target-part region and rejects only when the part is geometrically distorted beyond the intended motion, duplicated, or fails to move at all. If the critic rejects, we resample the editing model with the same instruction and re-evaluate, up to three attempts per joint. Joints that fail all three attempts are routed to the fallback path described below. On the PartNet-Mobility re-annotation, this gating accepts 74% of joints under the visual path, with 2.1 generation attempts per accepted joint on average; the remaining 26% are passed to the fallback.

Differentiable render-and-fit with multi-init sweep. Accepted target frames are paired with the source frame and the selected axis, and the scalar joint state is recovered by differentiable render-and-fit. Let $\theta \in \mathbb{R}$ denote the scalar joint state and let R_θ be a differentiable renderer [Ravi et al. 2020] that transforms the moving part by the joint configured at θ and emits a silhouette mask M_θ and a Z-buffer depth map D_θ. A central design choice in this stage is what supervision to extract from the synthesized target frame. The rendered source frame and the image-edited target frame share the joint configuration we want to recover but differ systematically in low-level appearance: lighting, shading, and shadow placement are subtly inconsistent between the renderer's flat materials and the editing model's photoreal-leaning output. A direct photometric loss between the two frames mixes the signal we care about (joint state) with appearance artifacts we do not, and in our preliminary experiments this loss did not converge reliably under reasonable optimizer settings. We instead rely on two appearance-invariant signals: a silhouette and a calibrated depth.

Silhouette via SAM-3 with cross-frame propagation. For the moving part's silhouette $\widehat{M}$ on the target frame we use SAM-3 [Carion et al. 2025] not as a single-image prompted segmenter but as a mask tracker. SAM-3 is initialized with the per-part label map of the source frame, which the rendering pipeline supplies directly rather than predicting, and is then asked to propagate every label through the two-frame video formed by source and target. This guarantees that the target mask returned for part X is the segmentation of the same physical part as in the source.

Metric depth via DA3 calibrated against the rendered source. For the depth map $\widehat{D}$ we use the relative-depth variant of Depth-Anything 3 [Lin et al. 2025]. Because relative depth is defined only up to an unknown scale and shift, we calibrate it against the source frame's rendered depth, whose values are exact because the camera's intrinsics and extrinsics are known by construction. We fit a single $(s, b) \in \mathbb{R}^2$ by least squares on the static-region pixels of the source, which depict the same surface in source and target by definition of the static region, and apply the same (s, b) to the target's relative depth to obtain a metric depth aligned with the renderer's Z-buffer.

The optimization then minimizes the weighted sum of a silhouette term and a depth term, with weights λ_m and λ_d:

$$\mathcal{L}(\theta) \;=\; \lambda_\mathrm{m}\,\mathrm{BCE}\big(M_\theta,\,\widehat{M}\big) \;+\; \lambda_\mathrm{d}\,\big\|(D_\theta - \widehat{D}) \odot (M_\theta \odot \widehat{M})\big\|_1, \qquad (1)$$

where BCE denotes pixel-wise binary cross-entropy, $\odot$ denotes the elementwise product, and the depth term is masked to the intersection of predicted and target silhouettes. This intersection masking prevents body-versus-target depth comparisons from flipping the pose when the rendered geometry happens to occupy the target region under a wrong θ. We use $\lambda_\mathrm{m} = 5$ and $\lambda_\mathrm{d} = 0.1$, optimise θ with Adam at learning rate $2{\times}10^{-2}$ for 200 iterations, and freeze every

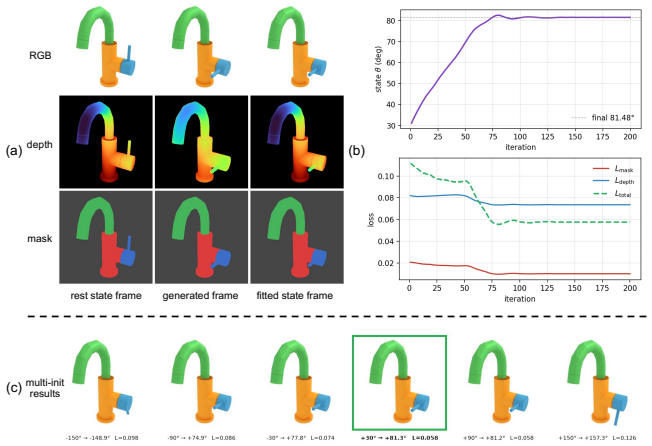


Fig. 9. End-to-end render-and-fit for a lever faucet handle. (a) RGB, calibrated metric depth, and SAM-3 part mask, shown for the rest-state render (left), the image-edited target frame (middle), and the fitted state recovered by minimising Eq. (1) (right). (b) Optimisation traces of the scalar joint state θ (top) and of the silhouette, depth, and combined losses (bottom) over the 200 Adam iterations. (c) Multi-init sweep on the same joint: each panel reports the initial state, the converged value, and the final loss, with the basin of lowest loss selected as the released estimate. The faucet also illustrates the semantic-sign argument of the previous discussion, since either direction of rotation yields a geometrically valid open state.

other parameter so that range recovery is one-dimensional. Renders use the source-frame resolution of 1024×768 and PyTorch3D's soft-silhouette rasterizer with a blur radius of 10^{-5} and 8 faces per pixel. This rasterizer setting keeps internal mesh seams off the silhouette-loss floor and lets gradients propagate cleanly across the moving part's boundary.

Single-frame fitting under a one-dimensional objective is prone to local minima: a partially open door admits both a small positive and a large negative θ that produce comparable silhouettes from a single viewpoint, and depth alone is rarely enough to break the tie. We mitigate this by sweeping a small grid of initial values that covers both motion directions and selecting the one with the lowest final loss. For finite-limit rotational joints we sweep $\theta_0 \in \{-150°, -90°, -30°, 30°, 90°, 150°\}$, and for translational joints (in normalized object units) $\theta_0 \in \{-0.5, -0.2, 0, 0.2, 0.5\}$. The grids are coarse by design: they exist only to place the optimizer in the correct basin of attraction, and the gradient term then localizes the limit precisely once that basin is fixed. Figure 9 shows a representative trajectory of this stage on a lever faucet handle.

VLM fallback with explicit sign cues. The fallback path is used for joints that the visual path does not estimate, either because the planner routes them to the fallback in the first place (e.g. small or repetitive controls such as panel buttons) or because every synthesized frame is rejected by the critic after the retry budget is exhausted. For each fallback joint we render the per-joint hint shown on the right of Figure 10 and prompt a range-specialist VLM with the hint, the joint type, and the selected axis direction using Prompt 7. For revolute joints, the curl visual makes the right-hand-rule sign explicit.



Fig. 10. Inputs used by the VLM fallback range estimator. Left: the global initial render supplied as object context. Right: the per-joint hint image generated after axis selection, with the moving display panel highlighted in red, the selected axis drawn as a directed yellow arrow, and a partial-torus curl visualising the right-hand-rule positive direction.

For prismatic and screw joints, the arrow direction fixes the sign convention. The model then returns a single signed target range together with a textual rationale.

After visual fitting and fallback estimation, we combine the selected axis with the signed range for each joint. Continuous joints keep the placeholder range $[0, 0]$. In the faucet example, the accepted edit for the handle converges to a positive rotation of 81.478°, yielding the final prediction below. Together with the part manifest, this record is directly convertible to and from a standard URDF file: manifest parts become links, parent assignments become joints, and finite revolute limits are written in radians.

Example merged motion output


```
[
  {
    "id": 0,
    "parent": 1,
    "joint": {
      "type": "revolute",
      "range": [0, 81.478],
      "axis": {
        "origin": [0.2644, -0.3026, -0.1357],
        "direction": [1, 0, 0]
      }
    }
  },
  {
    "id": 1,
    "parent": -1,
    "joint": {
      "type": "fixed",
      "range": [0, 0],
      "axis": {
        "origin": [0, 0, 0],
        "direction": [0, 0, 1]
      }
    }
  },
  {
    "id": 2,
    "parent": 1,
    "joint": {
      "type": "continuous",
      "range": [0, 0],
```

```
      "axis": {
        "origin": [-0.0923, 0.1886, -0.1356],
        "direction": [0, -1, 0]
      }
    }
  }
]
```


For the same prediction, an abbreviated URDF export has the following form.

Example URDF export


```
<robot name="faucet_2054">
  <link name="base_link"/>
  <link name="surface_base">
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry>
        <mesh filename="textured_objs/original-17.obj"/>
      </geometry>
    </visual>
    <!-- Full mesh list omitted. -->
  </link>
  <joint name="joint_1" type="fixed">
    <parent link="base_link"/>
    <child link="surface_base"/>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <axis xyz="0 0 1"/>
  </joint>
  <link name="switch">
    <visual>
      <origin xyz="-0.2644 0.3026 0.1357" rpy="0 0 0"/>
      <geometry>
        <mesh filename="textured_objs/original-105.obj"/>
      </geometry>
    </visual>
    <!-- Full mesh list omitted. -->
  </link>
  <joint name="joint_0" type="revolute">
    <parent link="surface_base"/>
    <child link="switch"/>
    <origin xyz="0.2644 -0.3026 -0.1357" rpy="0 0 0"/>
    <axis xyz="1 0 0"/>
    <limit lower="0.0" upper="1.4221" effort="10.0" velocity
      ="1.0"/>
  </joint>
  <link name="tube">
    <visual>
      <origin xyz="0.0923 -0.1886 0.1356" rpy="0 0 0"/>
      <geometry>
        <mesh filename="textured_objs/original-209.obj"/>
      </geometry>
    </visual>
    <!-- Full mesh list omitted. -->
  </link>
  <joint name="joint_2" type="continuous">
    <parent link="surface_base"/>
    <child link="tube"/>
    <origin xyz="-0.0923 0.1886 -0.1356" rpy="0 0 0"/>
    <axis xyz="0 -1 0"/>
  </joint>
</robot>
```


Quantitative range accuracy and limitations. On the same re-annotation, we score motion ranges against ground truth using the angular and translational error metrics defined in ArtiBench. Averaged over all non-fixed joints, the mean angular error is 28.4° and the mean translational error is 0.272 m. These numbers should be read in context: motion ranges in articulated-object datasets carry an irreducible amount of definitional slack. Drawers, for example, are sometimes annotated at the point of full extraction from the cabinet and sometimes at a partially-extended state in which the drawer remains visibly seated, and the same rear-hinged door may appear at 90° in one asset (the smallest angle that clears the opening) and at 110° in another (an angle that leaves practical reach into the cabinet). At this level of ambiguity, the absolute errors above are well within the spread we observe between different but equally defensible human annotations.

We acknowledge two distinct limitations that remain. First, image generation and VLM inference are not deterministic, and we observe occasional sign flips on assets where the right-hand-rule cue is visually subtle, for example revolute axes that are nearly parallel to the camera's view direction in the chosen orbital frame. Second, our pipeline currently simplifies range to a *one-sided* estimate: starting from the rest pose at $\theta = 0$, we synthesize or reason about a single target state at the opposite mechanical limit, so the released range takes the form $[0, x]$ or $[-x, 0]$ with $x > 0$. This is sufficient for assets whose rest pose already coincides with one mechanical limit, such as closed doors, fully-seated drawers, and neutral knobs, which covers the majority of our data. It is not sufficient when the rest state lies in the interior of a two-sided range, as is the case for doors or laptop screens captured in a partially open state in which the joint can move both forward and backward from the input pose. The pipeline reports only one of the two sides for such cases. Extending it to a two-sided estimate would require synthesizing two target states per joint rather than one.

D.4 Human review and annotation efficiency

Annotation and review are performed by professional 3D artists. Before entering production, each artist receives task-specific training and must pass a qualification test. Every released asset passes through two independent rounds of human inspection in a Blender review interface. The interface loads the predicted articulated representation, exposes one slider per joint, and overlays the selected axis as a colored oriented line. This allows reviewers to inspect the kinematic tree, drive each part through its predicted motion, and verify axis placement and motion limits in one interaction. Reviewers can edit any of the predicted fields, including parent assignment, joint type, axis parameters, and motion range, and re-export the corrected result.

Assets are discarded rather than corrected when the underlying mesh cannot support a physically plausible motion. A common example is a drawer whose body is not parallel to the cabinet front, so that even a geometrically reasonable translation axis causes immediate self-intersection. A second example surfaces only after the slider is opened in the interface: a part that reads as a drawer from the outside but is modelled without the inner container cavity its housing implies, so pulling it open exposes the surrounding cabinet

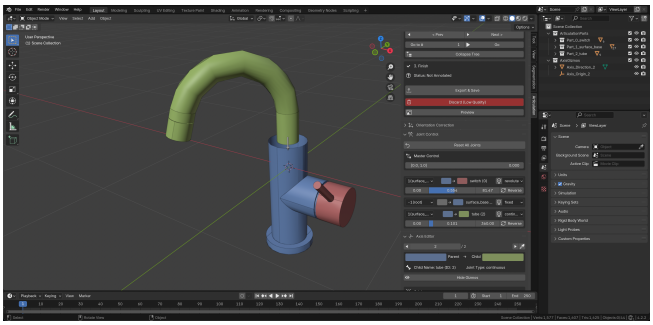


Fig. 11. Blender interface used for human review of articulation annotations. Annotators inspect the predicted articulated object, drive joints through their motion range, verify the overlaid axis, and either accept, correct, or discard the sample.

Perfect rate	Assets	Mean (s)	Median (s)	Joints/asset
= 0%	6,569	164	105	3.34
(0%, 25%)	178	374	323	9.00
[25%, 50%)	501	266	206	5.76
[50%, 75%)	597	242	173	6.22
[75%, 100%)	344	239	177	10.08
= 100%	3,995	69	36	3.43

Table 2. Per-asset annotation time on ArtiAtlas, bucketed by the asset's perfect rate, defined as the fraction of its non-fixed joints whose axis the reviewer accepted unchanged. Sessions with abnormally long idle gaps are filtered out before aggregation. *Joints/asset* reports the average number of non-fixed joints per asset within the bucket.

interior rather than a drawer body. Only assets that survive the second-pass review enter the final dataset.

Pipeline accuracy on ArtiAtlas. The main paper has established that ArtiAtlas covers a substantially broader range of categories than PartNet-Mobility, with a long tail that includes locks, robot arms, keyboards, and model weapons. Unlike the PartNet-Mobility evaluation above, ArtiAtlas has no pre-existing ground truth: the pipeline is the production annotator, and the released annotations are produced by the two-pass human review described above. We therefore treat those reviewed releases as the reference and score the pre-review predictions against them under the same per-stage metrics. Kinematic structure prediction reaches 97.2% parent assignment and 94.0% joint-type accuracy, within roughly one percentage point of the PartNet-Mobility numbers above. Axis estimation degrades more visibly: at the same 3° angular and 3% origin tolerance, the success rate drops from 80.5% to 61.4%, consistent with the broader catalogue of out-of-prototype geometries that ArtiAtlas introduces. Motion ranges remain comparable to PartNet-Mobility within the dataset's definitional slack, with mean angular error 34.3° and mean translational error 0.260 m.

Perfect rate and review-time savings. Stage-level accuracies do not directly capture the human effort the pipeline removes. We complement them with an end-to-end measurement collected from the review tool itself. After two-pass review, we compare the released axis against the prediction and call a joint *perfect* when the corrected axis is identical to the predicted one. The metric is restricted to axis edits because annotators consistently reported that the bulk of correction time is spent placing the 3D axis rather than adjusting the motion range. Across the ArtiAtlas review logs, 46.1% of joints are perfect under this criterion.

Per-asset timing logs, automatically recorded by the review tool and filtered for outlier sessions with long idle gaps, show a clear inverse relationship between an asset's perfect rate and its review time (Table 2). Assets whose joints are all perfect (=100%, mean 3.43 joints per asset) take an average of 69 s, while assets with no perfect joints (=0%, mean 3.34 joints, comparable in size) take 164 s, more than twice as long. The [75%, 100%) bucket sharpens the comparison further: it contains the most complex assets in the table, 10.08 joints per asset on average, exceeding the 6.22 joints of the [50%, 75%) bucket, yet its mean review time is lower (239 s vs 242 s) because most of its joints are accepted without edits.

These savings hold under a strict quality bar. Annotators are trained to align hinge axes exactly with the door's geometric edges, and to correct any visually detectable misalignment even when the predicted axis already produces a kinematically plausible motion. Even fully-perfect assets average more than a minute of review time, which reflects inspection cost rather than editing cost. The pipeline nonetheless removes a substantial fraction of the per-asset effort that fully manual kinematic fitting would incur on the harder ArtiAtlas assets.

D.5 Comparison with PhysXNet

Figure 12 compares random samples drawn from the PhysXNet [Cao et al. 2025a] kinematic subset (panel (a), 20 assets) and the ArtiAtlas training split (panel (b), 20 assets). Each cell shows the asset driven to its joints' lower limits (left) and upper limits (right), so that joint axis placement and motion range are visible together. The PhysXNet panel exhibits the characteristic failure modes of a semi-automated annotation pipeline: hinge axes are frequently flipped or offset away from the mechanical seam, causing doors and drawers to swing into the body or slide off-track, and the category mix is concentrated on storage furniture and seating. The ArtiAtlas panel covers a broader long-tail vocabulary including hand-held tools, musical instruments, machinery, packaging, and play objects, with joint axes consistently aligned with the visible articulation seam.

E Final Dataset Statistics and Release

E.1 Final composition and annotation-stage attrition

The filtering and annotation pipeline reduced the candidate pools to a 14,662-object pre-release pool: 11,049 assets from the multi-source raw-asset stream, 1,861 PartNeXt assets, and 1,752 PartVerse-XL assets. Final annotation-stage review retained 11,008, 1,827, and 1,721 assets from the respective streams. The last review stage excludes 106 objects and yields the reported 14,556-object dataset. These final counts, rather than the larger pools shipped to annotation, are used throughout the paper.

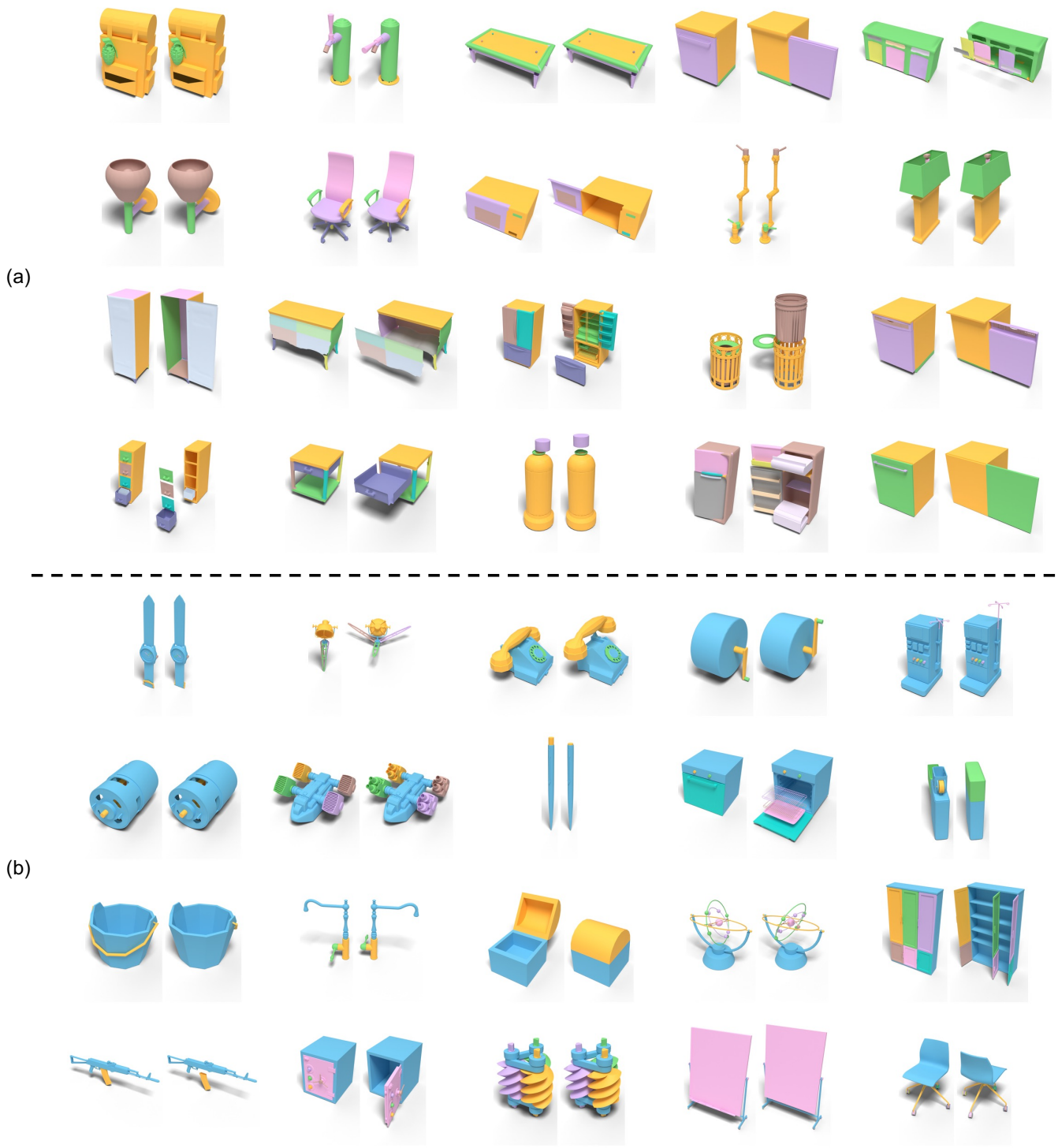


Fig. 12. **Random-sample comparison: PhysXNet vs. ArtiAtlas.** (a) twenty randomly sampled PhysXNet [Cao et al. 2025a] kinematic assets; (b) twenty randomly sampled ArtiAtlas training-split assets. Each cell shows the asset driven to its joints' lower limits (left) and upper limits (right). PhysXNet exhibits frequent axis flips, origin offsets from the part seam, and a category mix biased toward storage furniture and seating; ArtiAtlas samples cover a broader long-tail vocabulary with consistently aligned axes.

E.2 Category taxonomy and distribution

Controlled-vocabulary classification. Every asset carries a free-form object caption written by its human annotator during articulation annotation. We aggregate these captions across the corpus and cluster them to establish which concepts the data actually contains. Taking the raw captions as categories would split a single concept across near-synonyms and fragment the long tail, so we consolidate the clusters into a controlled vocabulary: we retain the 43 level-2 names shared with PartNet-Mobility [Xiang et al. 2020] and extend them to cover the long-tailed domestic, industrial, entertainment, and vehicle objects in ArtiAtlas, yielding 173 level-2 categories, each with a written scope definition. Only then is each asset labelled: a VLM cross-checks the rendered asset against the annotator's caption, proposes a label from the controlled vocabulary, and emits an explicit confidence and review status. Ambiguous or low-confidence cases are surfaced for human review.

The vocabulary uses category-specific definitions to separate labels that look similar in abbreviated plots. For example, *Screen* denotes room dividers, folding screens, and partitions, whereas *Monitor* denotes electronic displays and televisions. *Oven* denotes an enclosed baking or heating chamber, whereas *Stove* denotes a stovetop, cooktop, or range. The vocabulary also distinguishes toy-coded weapons from replicas and ordnance. *Toy Gun* covers water, airsoft, and foam-dart guns, whereas *Model Weapon* covers weapon replicas and non-toy ordnance models such as cannons, crossbows, grenades, and display firearms. The released taxonomy provides the definition and aliases associated with every controlled label.

Long-tail statistics. Per-category counts are given in Table 3. Category size ranges from two to 1,460 assets, with a median of 35 and a mean of 84.1. *Cabinet* is the largest category (1,460; 10.0%). The distribution remains deliberately long-tailed: 103 of 173 categories (59.5%) contain fewer than 50 assets and together account for 13.4% of the corpus, whereas the ten largest categories account for 40.2%.

E.3 Data sources and release contents

Composition of the multi-source stream. The final 11,008 assets in this stream comprise 9,721 objects from the ten research datasets in Table 4 and 1,287 objects obtained from a commercial partner. Together with the 1,827 PartNeXt and 1,721 PartVerse-XL assets, these sources yield the final 14,556-object dataset. We verified that the training pool does not overlap with PartNet-Mobility, Lightwheel, or the evaluation suites used in this paper through source-identifier and geometric near-duplicate checks.

Release contents. Gated release is the standard arrangement for 3D datasets assembled from license-restricted sources [Chang et al. 2015; Fu et al. 2021; Iliash et al. 2026b; Khanna et al. 2024], and we follow it: ArtiAtlas is distributed as a gated Hugging Face repository in which users acknowledge the licensing of the original data and request access for research use before downloading. The release contains complete meshes, link segmentations, and articulation annotations for 10,005 objects: 6,457 from the nine research datasets, 1,827 from PartNeXt, and 1,721 from PartVerse-XL. For the 3,264 objects drawn from Objaverse-XL we follow the upstream practice of releasing links rather than geometry, since Objaverse-XL itself

Category	Assets	Category	Assets
Abacus	4	Machine Tool	18
Action Figure	27	Measuring Tool	76
Air Conditioner	4	Mechanical Toy	9
Aircraft	32	Medical Instrument	21
Amusement Ride	17	Microphone	28
Antenna	47	Microscope	44
Barbell	15	Microwave	80
Basket	20	Mine Cart	18
Bed	97	Mirror	120
Bicycle	2	Miscellaneous Mechanism	166
Board	77	Model Weapon	254
Boat	8	Monitor	143
Book	23	Mop	4
Bottle	327	Motorcycle	6
Box	708	Mount	39
Bucket	222	Mouse	29
Building	19	Nail Gun	3
Cabinet	1,460	Ornament	47
Cage	9	Oven	83
Calculator	5	Paint Roller	18
Calendar	4	Pen	62
Camera	164	Phone	105
Car	68	Photo Frame	29
Cart	411	Piano	38
Caster	41	Pliers	59
Chair	455	Pressure Vessel	3
Clamp	178	Printer	31
Clock	83	Projector	12
Clothespin	13	Pump	23
Coffee Machine	66	Radio	44
Coffin	9	Record Player	47
Connector	12	Refrigerator	89
Control Panel	41	Remote	32
Conveyor Belt	13	Ride Toy	9
Crane	23	Robot	154
Cup	16	Robotic Arm	207
Dishwasher	13	Rod	46
Dispenser	198	Router	14
Door	414	Safe	79
Dresser	45	Saw	36
Drill	42	Scissors	71
Drum	2	Screen	217
Easel	20	Sewing Machine	3
Elevator	6	Shelf	59
Engine	77	Shower	15
Excavator	14	Sink	48
Eyeglasses	101	Skateboard	101
Fan	222	Sofa	17
Faucet	380	Solar Panel	15
Fidget Toy	6	Speaker	15
Fire Extinguisher	52	Stapler	14
Fire Hydrant	17	Stove	58
Fishing Reel	2	Stroller	9
Flashlight	75	Submarine	2
Folding Chair	33	Suitcase	14
Forklift	22	Swing	35
Game Console	110	Switch	184
Gear	142	Syringe	21
Globe	120	Table	446
Grill	32	Tank	60
Grinder	21	Tape Dispenser	11
Guitar	77	Teapot	108
Hair Dryer	3	Telescope	52
Handle	108	Toaster	30
Harvester	8	Toilet	106
Headphone	113	Toy Gun	24
Helmet	13	TP Holder	18
Hinge	93	Tractor	9
Hoist	17	Train	4
Iron	2	Trap	8
Jack	25	Trash Can	214
Kettle	67	Tripod	34
Keyboard	145	Typewriter	5
Kitchen Appliance	75	Umbrella	6
Knife	73	USB	6
Lab Apparatus	15	Vacuum Cleaner	10
Ladder	40	Valve	143
Lamp	917	Vehicle Part	11
Laptop	139	Vending Machine	4
Lawn Mower	10	Wardrobe	86
Lighter	37	Washing Machine	60
Linear Actuator	29	Watch	59
Linear Stage	14	Well	66
Linkage	337	Windmill	58
Lipstick	15	Window	224
Lock	140	Wrench	68
Loom	5		

Table 3. Per-category composition of ArtiAtlas.

Source	Venue	License	Assets
3D-FUTURE [Fu et al. 2021]	IJCV 2021	Custom research-only agreement	273
3DCoMPaT200 [Ahmed et al. 2024]	NeurIPS D&B 2024	CC BY 4.0	640
ABC [Koch et al. 2019]	CVPR 2019	MIT	1,422
AI2-THOR [Kolve et al. 2017]	arXiv 2017	Apache-2.0	137
FG3D [Liu et al. 2021]	IEEE TIP 2021	Upstream terms‡	41
Fusion 360 Gallery [Willis et al. 2021]	ACM TOG 2021	Custom non-commercial research	250
HSSD [Khanna et al. 2024]	CVPR 2024	CC BY-NC 4.0	1,001
ModelNet [Wu et al. 2015]	CVPR 2015	Research use only‡	889
Objaverse-XL [Deitke et al. 2023]	NeurIPS D&B 2023	ODC-By 1.0§	3,264
ShapeNetCore [Chang et al. 2015]	Tech. report 2015	ShapeNet Terms of Use	1,804
Total			9,721

Table 4. Composition of the public-research-dataset portion of the multi-source raw-asset stream, with the publication venue and the distribution license of each source. ‡ No license document is published: ModelNet's download page states academic research use, and both sources collect their shapes from third-party repositories whose terms continue to apply. § The license covers the index; each indexed object carries the license of its own source.

indexes assets by source URL; the entries used here resolve to 2,304 GitHub and 960 Sketchfab objects [Deitke et al. 2023]. For these objects we provide the Objaverse-XL identifiers, per-link `face_ids`, articulation annotations, and scripts that reconstruct the full articulated assets from the indexed source meshes.

F Implementation Details

This appendix collects the implementation choices behind Artisan (Section 4) and the comparison methods evaluated alongside it in Section 6.

F.1 Artisan architecture and training

Decoder. The decoder is a six-layer bidirectional transformer with dropout 0.1. Each layer first updates the part queries with self-attention and query-to-point cross-attention, then updates the point tokens with point-to-query cross-attention. Mask-aware attention is applied only in the query-to-point direction: the first layer attends to every point, and from layer two onwards we mask out any point whose sigmoid probability under the previous layer's mask logits falls below 0.5. A row that would otherwise mask every point is reset to unmasked, so a query is never starved of input. Mask logits themselves are dot products between projected query and point tokens, computed in chunks of 2048 points to bound activation memory. The first five layers also expose intermediate class and mask heads for auxiliary supervision.

Prediction heads. Parallel heads read the final decoder output. A two-way classifier predicts whether each query is present in the assembly, a parent-relation MLP scores ordered query pairs, and a four-way classifier assigns each query a fixed, revolute, prismatic, or screw label. The revolute and prismatic motion heads share an output shape and each emits a 3D axis direction with the two scalars that bound the range; screw joints use both. A per-point head takes each surface point together with its assigned query token and regresses the orthogonal projection of that point onto its link's revolute axis; the median of these votes over a link's points is the pivot exported to the URDF. A query-level axis-pivot head, a two-layer MLP on the query embedding, regresses the same median and is used only during training.

Losses. Training follows a DETR-style set-prediction objective. For each shape with P ground-truth links and M predicted queries, we first compute a Hungarian matching from links to distinct queries. The matching cost between query j and ground-truth link p uses only the present/no-object classification and mask terms:

$$C_{jp} = 2.0\, C_{jp}^{\text{present}} + 5.0\, C_{jp}^{\text{bce}} + 2.0\, C_{jp}^{\text{dice}}. \tag{2}$$

The BCE and Dice mask costs are normalized by mask area. Unmatched queries are supervised as no-object, with the no-object coefficient set to 0.1.

After matching, each matched query receives link-level and motion supervision. The final-layer objective is

$$\begin{aligned}\mathcal{L} = {} & \lambda_{\text{query}}\mathcal{L}_{\text{query}} + \lambda_{\text{mask}}\mathcal{L}_{\text{mask}} + \lambda_{\text{tree}}\mathcal{L}_{\text{tree}} \\ & + \lambda_{\text{type}}\mathcal{L}_{\text{type}} + \lambda_{\text{motion}}\mathcal{L}_{\text{motion}} + \lambda_{\text{aux}}\mathcal{L}_{\text{aux}}.\end{aligned} \tag{3}$$

Here $\mathcal{L}_{\text{query}}$ is cross-entropy over present/no-object labels, and $\mathcal{L}_{\text{mask}}$ is the sum of sigmoid BCE and Dice losses on per-point mask logits. $\mathcal{L}_{\text{tree}}$ is class-balanced binary cross-entropy over ordered parent-child query pairs, and $\mathcal{L}_{\text{type}}$ is cross-entropy over fixed, revolute, prismatic, and screw joint labels.

The motion loss is applied only to matched queries and only at the final decoder layer:

$$\begin{aligned}\mathcal{L}_{\text{motion}} = {} & 2.0\,\mathcal{L}^{r}_{\text{axis}} + 0.1\,\mathcal{L}^{r}_{\text{range}} + 1.0\,\mathcal{L}^{p}_{\text{axis}} \\ & + 1.0\,\mathcal{L}^{p}_{\text{range}} + 2.0\,\mathcal{L}_{\text{closest}} + 1.0\,\mathcal{L}_{\text{pivot}}.\end{aligned} \tag{4}$$

All terms are L1 losses. $\mathcal{L}^{r}_{\text{axis}}$ and $\mathcal{L}^{r}_{\text{range}}$ supervise revolute axis direction and angular range, while $\mathcal{L}^{p}_{\text{axis}}$ and $\mathcal{L}^{p}_{\text{range}}$ supervise prismatic direction and translation range. Continuous joints are encoded as revolute targets with a 360° range. Screw joints contribute to both the revolute and prismatic terms, and fixed joints do not contribute to continuous motion losses. The revolute-axis and closest-axis terms are upweighted because pivot localization is otherwise weakly constrained, while the revolute-range term is downweighted to avoid small-angle errors dominating the gradient.

For the closest-axis loss, each sampled point on a revolute link is assigned the orthogonal projection of that point onto the ground-truth revolute axis. The per-point head regresses this projected 3D point. The query-level axis-pivot target is the median of these projected points over all sampled points assigned to the link. This gives the model both dense point-level supervision and a direct link-level pivot signal.

The auxiliary loss $\mathcal{L}_{\text{aux}}$ repeats the query cross-entropy and mask BCE/Dice losses on the first five decoder layers, each with weight 0.25. All other final-layer loss weights are 1.0 unless specified above: query classification, mask losses, parent-relation loss, joint-type loss, prismatic-axis loss, prismatic-range loss, and query-level axis-pivot loss use weight 1.0.

Axis-based joints have a sign ambiguity:

$$(\mathbf{u}, [r_{\min}, r_{\max}]) \equiv (-\mathbf{u}, [-r_{\max}, -r_{\min}]). \tag{5}$$

We canonicalize every nonzero axis before constructing motion targets. Let $k = \arg\max_d |u_d|$ be the dominant coordinate of the axis. If $u_k < 0$, we flip the axis sign and swap the range according to Eq. 5; otherwise the label is unchanged. The canonicalized axis is then used to derive the closest-axis and axis-pivot targets, ensuring that all motion heads share the same deterministic sign convention.

Optimisation and inference. We train with AdamW at $\beta = (0.9, 0.999)$, weight decay 0.01, $\epsilon = 10^{-8}$, and per-process batch size 4 without gradient accumulation. The learning rate warms up linearly over 2,000 steps to the peak value used in Section 6.1 and then follows a cosine schedule down to one-tenth of that peak, with gradient norms clipped at 1.0 throughout. At inference, every Artisan variant uses the same deployable cardinality policy: a query is retained when its present-times-quality score reaches 0.20, with a top-one fallback if no query passes the threshold.

F.2 Comparison methods

We run every comparison method through its official source code and released checkpoint, with the single exception of Real2Code [Mandi et al. 2024], which is re-trained from scratch because no weights ship with the code. For 3D-input methods (Particulate [Li et al. 2025], URDF-Anything [Li et al. 2026]), we additionally verify that the point cloud sampled from the input mesh respects the normalisation convention and canonical orientation expected by the released checkpoint.

Every comparison method bounds how many parts it can emit, whether by a hard part limit (Particulate's 16 parts, or the 32 slots of SINGAPO [Liu et al. 2025a]), by a prompt or part vocabulary that admits only a handful of parts (Articulate-Anything [Le et al. 2025], URDFormer [Chen et al. 2024]), or by a token or voxel budget (URDF-Anything [Li et al. 2026], Real2Code [Mandi et al. 2024], the PhysX models [Cao et al. 2025a,b]). These bounds bind well below our data: gt objects in the Category-Rich Suite have a median of four parts and a ninetieth percentile of 13, whereas the baselines predict a median of one to three parts and a ninetieth percentile of at most 9. We run each method at its native capacity and keep the complete gt object during evaluation, so links beyond that capacity remain unmatched and receive the penalties defined in Appendix G.2.

Unified VLM backend and auxiliary inputs. Several pipelines do not consume their primary modality in isolation. SINGAPO [Liu et al. 2025a] requires a part graph extracted by an external VLM, and Articulate-Anything [Le et al. 2025] drives every stage from VLM agents. To keep this variable controlled across methods, we route both backends through Gemini-3-Flash, the same model that powers our own annotation pipeline (Appendix D). For methods whose input also includes an explicit part count or an object category (URDF-Anything, SINGAPO, URDFormer [Chen et al. 2024]), we supply the dataset's ground-truth values on the Category-Rich Suite and call Gemini-3-Flash to extract the same fields from the photograph on the In-the-Wild Suite, mirroring how a user would deploy these methods on novel inputs. When a method exposes a closed category vocabulary, we ask the VLM to map the input to the closest supported label; assets whose true category lies outside that vocabulary are still fed in under the closest mapping, and any resulting degradation (for example, a furniture cabinet returned for an unrelated input) is reported as-is.

Per-part image segmentation. Two image-input methods, PAct [Liu et al. 2026] and URDFormer, consume per-part image labels: PAct expects a part-id mask aligned with the input render, and URDFormer expects per-category bounding boxes that its released pipeline obtains through GroundingDINO. On the Category-Rich Suite, we render the part-id mask from the same view used to condition each method and derive URDFormer's bounding boxes from that mask, so the part-level evidence presented to the model is exact. On the In-the-Wild Suite, where no 3D reference exists, we first prompt Gemini-3-Flash to enumerate the part names visible in the photograph and then run SAM-3 [Carion et al. 2025] with each name as a text prompt to recover per-part masks, from which URDFormer's bounding boxes are derived.

Articulate-Anything: gallery and Part-mesh setting. Articulate-Anything was released with an image and a video pipeline, both of which retrieve each candidate mesh from a fixed PartNet-Mobility gallery [Xiang et al. 2020]. Restricting the gallery to its original contents would systematically disadvantage the method on ArtiAtlas, where many of our categories lie outside PartNet-Mobility. We therefore expand the gallery to the full ArtiAtlas training split of 14,356 assets after holding out the 200 Category-Rich Suite samples. Each asset carries a CLIP ViT-B/32 image embedding so that the retrieval stage operates over an order-of-magnitude larger candidate pool than the released setup.

The released selector performs a hierarchical VLM tournament: it submits every shortlisted candidate to VLM, alongside the query, in batches and rounds. On the expanded gallery this becomes intractable in the long-tailed categories that dominate the in-the-wild suite (for instance the *Cabinet* class contributes 1,455 training assets), inflating both latency and API cost by more than an order of magnitude. We therefore insert a CLIP pre-filter ahead of the tournament: candidate image embeddings are compared to the query embedding and the top-50 are retained, after which the original VLM-driven tournament proceeds unchanged on the shortlist.

Articulate-Anything's Part-mesh setting corresponds to feeding the joint-prediction actor a ground-truth part decomposition in place of the upstream mesh-retrieval and link-placement outputs.

We pass the test sample's pre-decomposed OBJ parts directly into Articulate-Anything's `odio_urdf` API and run the unmodified joint-prediction actor over them, with the link summary built from the test sample's semantic part manifest. The actor uses the released image pipeline's in-context exemplars and actor-only inference policy, with the sampling temperature set to 1.0 as required by Gemini-3.

Particulate reproduction. The *Particulate (re-impl.)* row in Table 8 is our own training run of Particulate [Li et al. 2025]. The original authors granted access to their internal training repository and confirmed the configuration of the released run in private correspondence, including a mixed PartNet-Mobility [Xiang et al. 2020] and GRScenes [Wang et al. 2024] training corpus, global batch size 128, peak learning rate 6.4×10^{-5}, a $\leq$ 16-part filter on the training shapes, and a +Z-up canonicalisation. We follow the authors' checkpoint-selection protocol of taking the best snapshot from $\{20, 40, 60, 80, 100\}$k iterations per run, since some training runs plateau before 100k. After multiple training attempts against this configuration the reproduction still trails the released checkpoint by a wide margin on most metrics, and we have not been able to close the gap with the public information available at submission. We retain the *(re-impl.)* row as a reproducibility lower bound and report the released-checkpoint result as the primary Particulate number throughout the paper. The *Particulate (ArtiAtlas-trained)* rows in Tables 2 and 4 of the main paper use this same implementation, trained on the ArtiAtlas training split for 200K steps to match the training split and optimization budget of Artisan while retaining Particulate's native architecture.

Real2Code reproduction. Real2Code releases code but no trained weights, so we reproduce the bounding-box variant following the adaptation that Articulate-Anything uses for its Real2Code baseline. The model is fine-tuned from CodeLlama-7B-Instruct [Roziere et al. 2023] with QLoRA [Dettmers et al. 2023] on the PartNet-Mobility training split, matching the data scope of the original Real2Code release. We use rank-16 LoRA adapters with $\alpha = 32$ and dropout 0.05, train for three epochs with AdamW at learning rate 10^{-4} under half-precision mixed precision, an effective batch size of 16 (per-device batch 4 with 4 gradient-accumulation steps), and a maximum sequence length of 1024 tokens. The input serialisation follows Real2Code's relative-OBB mode: oriented bounding boxes are canonicalised against the root part and serialised as Python-style dictionaries, and the target is a list of child-joint dictionaries with joint type, principal-axis index, OBB-edge anchor, and sign. The same parsing convention is used at inference.

F.3 Connectivity-based label refinement

Many handcrafted 3D assets carry mesh-level connected components that already track their physical part decomposition, so mesh connectivity has long been used as a post-hoc prior on top of point-level part predictions. Both Particulate [Li et al. 2025] and PartField [Liu et al. 2025b] include such a connectivity-based cleanup in their inference pipeline, and we follow the same practice for the whole-mesh-to-URDF methods evaluated here (Particulate, URDF-Anything, and Artisan).

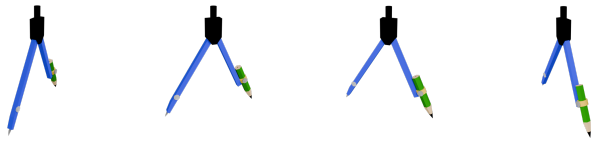


Fig. 13. **Canonical prompt views.** Each Category-Rich Suite object is rendered from four front-facing views and the conditioning image is sampled from these candidates with a balanced view distribution.

Without the cleanup, each face label is decided by majority vote over the sampled points that fall on it, and any face left unsampled adopts the label of its nearest sampled neighbour under face-centroid distance. The resulting labels reflect only the model's point-level prediction, without any topology signal.

Our post-processing then considers two variants of a connectivity-aware refinement step run on the mesh's face-adjacency graph. The strict variant treats each mesh connected component as a single rigid piece, computing the area-weighted dominant predicted label inside the component and overwriting every face with that label. The non-strict variant only requires that each predicted part be spatially contiguous within each mesh component, and sub-regions that share a label but are disconnected within their component are iteratively reassigned to the largest adjacent label until convergence.

The two variants are appropriate in different input regimes. Category-Rich Suite meshes are constructed by human artists, so their connectivity already tracks the physical part decomposition and the strict variant aligns the prediction with that prior. In-the-Wild Suite meshes are produced by an image-to-3D generator and are typically a single connected surface. Applying the strict variant in that regime would collapse the entire object to one rigid link, so we report the non-strict variant on the In-the-Wild Suite.

G Benchmark Details

This appendix expands the evaluation choices in Section 5: how prompt views are sampled, how the paired and reference-free metrics are defined, how missing predictions are aggregated, the diagnostic renders consumed by the VLM judge, and the human study that calibrates it.

G.1 Conditioning Images and Sources

Each Category-Rich Suite object is rendered from four front-facing canonical views (Figure 13). The conditioning image for an image-input method is drawn from these four candidates with a balanced split of 50 samples per view across the 200-object suite, so that image methods are not evaluated under a single camera pose. The In-the-Wild Suite uses 100 real photographs across 50 categories, with two images per category, from copyright-compatible sources; we deliberately select images that expose useful articulation evidence, such as an open appliance door or a deployed hinge, and pair each photograph with a foreground mesh produced by a commercial image-to-3D model.

G.2 Paired Metrics

Paired metrics require an annotated 3D reference and are reported only on the Category-Rich Suite.

Object geometry. Predicted and reference URDFs are assembled with the same forward-kinematics convention at the rest state and compared by whole-object Chamfer distance (R-obj CD).

Part decomposition. Predicted and reference links are matched by Hungarian assignment, after which we report rest-state per-part Chamfer distance and generalized IoU [Rezatofighi et al. 2019]. Following the penalized part-geometry protocol of Particulate [Li et al. 2025], unmatched links, whether they are extra predictions or missed reference parts, receive the worst per-part score, so over- and under-segmentation cannot hide behind matched pairs.

Joint matching and type acceptance. Non-fixed joints from the prediction and the reference are matched by Hungarian assignment on the rest-state bounding-box centers of their child links. We adopt the type-axis-origin success structure implemented by Articulate-Anything [Le et al. 2025] and related protocols in OPD, S2O, and URDF-Anything [Iliash et al. 2026a; Jiang et al. 2022; Li et al. 2026], replacing name-based correspondence with this geometric matching.

Type matching treats identical joint types as correct, with one asymmetric relaxation: a predicted revolute joint with range $\geq 330^\circ$ is accepted against a continuous reference, since several baselines do not separately model continuous joints and instead emit a wide-range revolute for spinning parts such as wheels or knobs.

Joint success. Axis error is the unsigned angular distance between matched world-frame axes. For revolute, continuous, and screw joints, origin error is the distance from the predicted origin to the reference joint-axis line; for prismatic joints the origin is a gauge variable and is excluded from the origin gate. A matched joint succeeds when (i) its type is correct under the rule above, (ii) its axis error is at most 0.25 radians, and (iii) its origin error, when defined, is at most 3% of the smaller non-zero world-frame bounding-box diagonal of the reference parent and child links, with a 0.05 m fallback when local geometry is unavailable. The scale-relative origin gate avoids imposing one world-space tolerance on parts of very different sizes. Missing reference joints and extra predicted joints count as failures. The resulting joint-success rate (*j-succ.*) subsumes type accuracy in our main result tables. Limit error is reported separately, since it captures motion range rather than kinematic correctness. Prismatic-plus-continuous helper chains are collapsed into a single logical screw joint before matching.

Origin-threshold sensitivity. We isolate the origin gate by holding predictions, joint matching, type decisions, and axis errors fixed, then re-gating the saved per-joint results, averaging within each object and macro-averaging over the 200 Category-Rich Suite objects. Under fixed absolute gates of 0.02, 0.05, and 0.10 m, joint success is 0.262, 0.356, and 0.398; under scale-relative gates of 1.5%, 3%, and 6% of the local bounding-box diagonal it is 0.219, 0.297, and 0.352. The two families differ less in level than in which errors they accept at a given part scale. Because a displaced origin is invisible at the rest pose, Figure 14 drives three joints away from rest about an axis displaced by the largest error each gate admits. The legacy 0.05 m equals 2.4% of the local diagonal for the door, 7.5% for the trash-can wheel, and 42.6% for the microscope button, so it grows more permissive as parts shrink; for the button, it accepts a prediction that swings it clear of its housing. Conversely, it rejects the door's 3% offset because that offset measures 0.063 m. A gate at 3% of the local diagonal accepts errors of comparable visual severity at every scale.

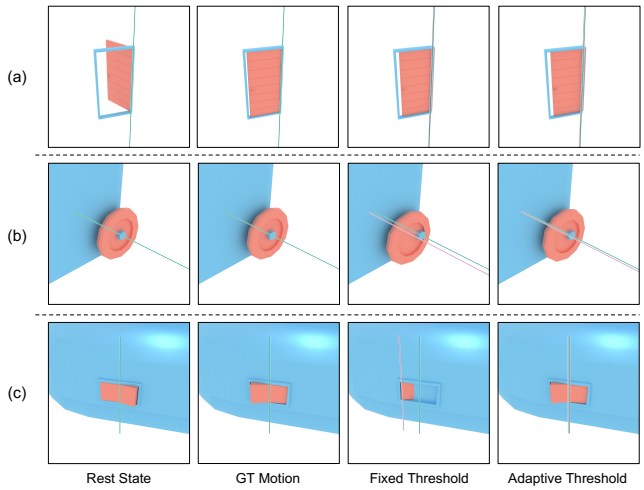


Fig. 14. **What each origin gate accepts across local part scales.** Rows (a)–(c): a door swung shut, a trash-can wheel spun half a turn, and a microscope button pressed to its limit, with local diagonals $D = 2.11$, 0.67, and 0.12 m. Columns: the rest state, the joint driven about the reference axis (green), and the same motion driven about an axis displaced perpendicular to it (magenta) by the largest error each gate admits, a fixed 0.05 m and 3% of D. Parent and child links are blue and red.

Articulated-state geometry. Following Particulate [Li et al. 2025], we sweep both objects through matched motion states and report articulated per-part Chamfer distance and gIoU (A-part CD / A-part gIoU). Axis directions are canonicalized for the articulated-state pass so equivalent sign conventions do not change the measured motion.

G.3 Reference-Free Metrics

Reference-free checks provide deterministic signals on both suites and are essential on the In-the-Wild Suite, where no 3D reference exists.

Kinematic-tree validation. The predicted URDF must define a connected acyclic kinematic graph: every joint references an existing parent and child link, and no link is orphaned, duplicated, or part of a cycle. We compute the per-sample validity rate but omit it from the main result tables because on ArtiBench this signal saturates: every evaluated baseline reports a validity rate between 88% and 100%, and the majority sit at 100%. The few below-100% baselines fail through pipeline-specific assembly artefacts rather than through deep-structure mistakes. Articulate-Anything [Le et al. 2025] emits the URDF as VLM-generated code and occasionally produces a document whose joints reference a link the same response forgot to

declare, which happens on 12% of part-mesh-input samples and 2.5% of image-input samples. URDFormer [Chen et al. 2024] relies on a discrete parent-relation head that, for each detected part, predicts an integer parent index; on roughly 12% of samples this head lands on an out-of-range value, and the implementation's fallback writes the part out as a link without instantiating a joint, leaving a detected handle or knob orphan. The remaining baselines reach 100% either by construction, where several hard-code a flat star around a single root [Li et al. 2026; Liu et al. 2026; Mandi et al. 2024], or in practice, because they predominantly saw shallow trees in PartNet-Mobility training and tend to predict similarly shallow structures, which still count as valid as long as they are well-formed. Since the check only tests structural well-formedness, it cannot distinguish methods that recover ArtiAtlas's deeper kinematic chains from those that simply flatten them out, and we treat it as a sanity check rather than a quality signal.

Self-overlap. We measure whether predicted links interpenetrate at rest (*R-overlap*) and under motion (*A-overlap*) with signed-distance queries rather than simulator contact, which is unreliable for concave cavities and intentional containment. For a mesh pair (A, B) the evaluator samples 50,000 volume points in total (half from each mesh) and defines $r_{A\to B}$ as the fraction of A-points inside B (and $r_{B\to A}$ analogously); the pair overlap ratio is $\max(r_{A\to B}, r_{B\to A})$, which penalises one part being embedded in another without depending on which contributes more volume.

R-overlap aggregates this ratio over all link pairs at the rest pose, taking the maximum pair ratio. A-overlap aggregates over *connected parent–child* pairs at sampled motion states: each 1-DoF joint is sampled at three deterministic states across its range, continuous joints over $[0, 2\pi)$, and helper-link chains use a Cartesian product capped at 64 configurations while preserving extreme states. We report the mean over pair–state combinations.

Signed-distance queries use CUDA BVH acceleration via cubvh, with coarse AABB tests skipping clearly disjoint pairs. The overlap metric depends on the mesh having a usable inside–outside signal: large holes, thin shells, or severe non-manifold artifacts can collapse the SDF sign and drive overlap toward zero even when the visible object is invalid. This is one of the reasons we additionally rely on the VLM judge for visible plausibility (Section 5.3).

G.4 Worst-fill Aggregation Under Missing Predictions

On a subset of hard samples some baselines cannot produce a result at all. URDF-Anything and PAct require the part count up front and silently truncate or refuse to decode when an object's true part count exceeds the model's configured capacity. Articulate-Anything relies on its VLM to emit a syntactically valid URDF document, and a handful of samples never yield a parseable XML even after three retries.

ArtiBench aggregates per-metric means with a *worst-fill* policy: for each sample and each metric, a missing prediction is charged the worst value that any other method produces on the same sample for the same metric. A method's reported number therefore reflects the cost of *not predicting* the hard cases, anchored to what the rest of the field could achieve there. The same worst-fill values determine the per-column "best" bolding in the experiment tables.

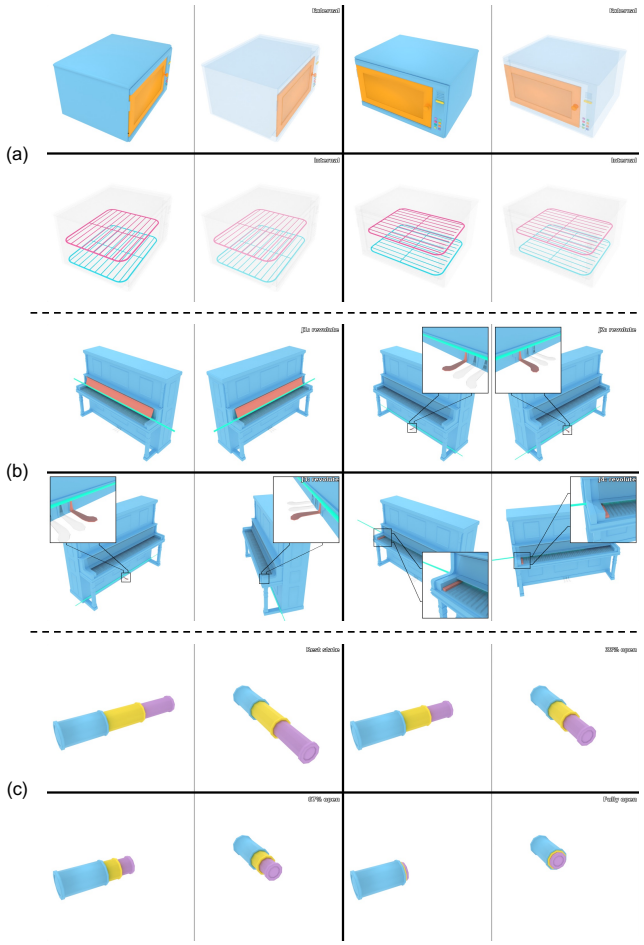


Fig. 15. **Diagnostic renders for VLM judging.** From top to bottom: (a) part-colored external/internal views for part decomposition and completeness, (b) joint-highlight views with axis overlays for joint type and joint axis, and (c) motion frames for articulation quality. Each render set is paired with the conditioning image and the aspect-specific rubric in the VLM judge.

G.5 VLM Diagnostic Renders

The VLM judge receives diagnostic renders tailored to each scored aspect. Part decomposition and per-part completeness use part-colored renders split into external and internal views via line-of-sight occlusion, so enclosed components such as trays, racks, or drums can be inspected without the model having to reason through an opaque shell. Motion quality is shown as a short rest-to-open sequence. Joint type and joint axis are shown as highlighted parent–child renders with axis overlays. When an object has many actuated joints, ArtiBench ranks joints by a weighted-volume importance score and displays at most four highlighted joint images, which keeps the multimodal context tractable [Wang et al. 2025b]. The prompt also includes the total actuated-joint count and a text summary of joint-type composition. The shared system prompt and per-aspect rubrics are reproduced in Appendix I.2.

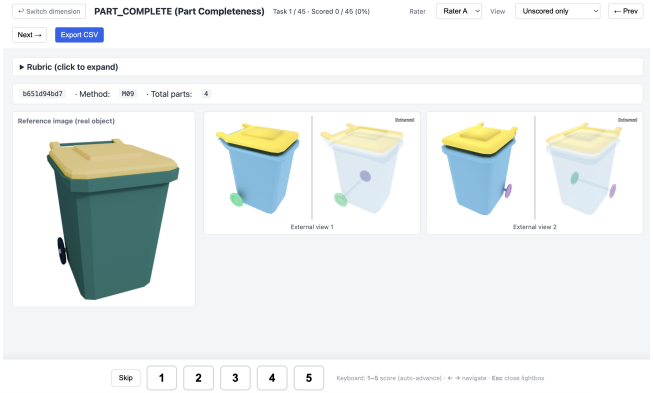


Fig. 16. **Human scoring interface.** The browser tool shows the reference image, the diagnostic renders for the current aspect, a hint bar with the anonymized method ID and aspect-relevant counts, the rubric (collapsible), and a 1–5 score control.

G.6 Human Preference Analysis

Protocol. Several expert evaluators with 3D research backgrounds manually scored articulated assets from 11 methods on 30 reference objects across the five aspects defined in Section 5.3, yielding 1,470 unique scoring tasks. PhysX-3D is omitted from the study because its outputs on the sampled 30 objects are predominantly rigid (no articulated joints recovered), leaving too few non-degenerate samples to score reliably. Method names were anonymized in the interface (Figure 16) and revealed only after every submission was collected.

Sample-level alignment. Table 5 reports Spearman's ρ between each metric and the per-aspect human score, comparing the VLM judge against the strongest deterministic metric for that aspect; lower-is-better metrics are negated, so positive values always indicate alignment with human preference. The VLM judge ranks first on every aspect with $\rho \geq 0.80$. The gap to the best deterministic proxy is small when a tight geometric correlate exists (MOTION: +0.05 over articulated per-part gIoU) and large otherwise, peaking at +0.21 on JOINT_AXIS over axis error and +0.15 on JOINT_TYPE over type accuracy. These gaps reflect that the deterministic counterparts are computed only on Hungarian-matched non-fixed joints, so a method that omits or mislabels actuated joints can show a small numerical error while the human still observes a structurally wrong object.

System-level alignment. Per-method mean human and VLM scores agree closely (Table 6): leaderboard Spearman is at least 0.95 on every aspect, and the top-ranked method coincides under both judges across all five aspects.

Robustness across repeated runs and judge backbones. We re-score a fixed set of 200 Category-Rich Suite outputs with the benchmark's diagnostic renders and rubrics. Across five recorded Gemini-3.1-Pro runs, the composite dataset mean has a standard deviation of 0.017 on the 1–5 scale, and the largest range of any per-aspect mean is 0.14. For the cross-backbone check, Gemini-3.5-Flash and GPT-5.5 change the composite by −0.03 and +0.11, with per-sample composite Pearson correlations of $r = 0.90$ and 0.83, respectively. These checks support stability after composite aggregation, although individual aspect scores are not numerically interchangeable across judge backbones.

	Part Decomp	Part Compl	Motion	Joint Type	Joint Axis
count_acc	0.402	0.416	–	–	–
CD (per-part)	0.697	0.724	0.720	–	–
mIoU (per-part)	0.750	0.714	0.756	–	–
gIoU (per-part)	0.762	0.748	0.758	–	–
limit_err	–	–	0.606	–	–
joint_success	–	–	0.643	0.668	0.598
type_acc	–	–	–	0.723	–
axis_err	–	–	–	–	0.624
origin_err	–	–	–	–	0.575
VLM (ours)	**0.874**	**0.824**	**0.805**	**0.870**	**0.837**

Table 5. **Sample-level Spearman ρ with human preference.** VLM judge vs. the strongest deterministic metric available for each aspect. Lower-is-better metrics are negated; bold marks the per-aspect winner.

Aspect	Pearson r	Spearman ρ	Kendall τ_b	Top-1	Top-3 set
Part Decomp	0.985	0.976	0.929	match	100%
Part Compl	0.990	0.952	0.857	match	100%
Motion	0.960	0.954	0.852	match	100%
Joint Type	0.969	0.961	0.881	match	100%
Joint Axis	0.981	0.973	0.927	match	100%

Table 6. **System-level VLM–human alignment** across per-method means ($n = 8$ methods for the two part aspects, $n = 11$ for the others). Top-1 indicates whether the top-ranked method coincides between the two judges; Top-3 set is the fraction of methods present in both top-three lists.

G.7 Test-set Gallery

Figure 18 shows representative objects drawn from the three evaluation pools used in this paper. Panel (a) samples the Category-Rich Suite of ArtiAtlas: ten held-out assets that span everyday articulated objects (basket, recycling bin, ladder, microphone stand) together with long-tail entries (binder clip, robot arm, action figure, router, skateboard) that are not covered by prior benchmarks. Panel (b) samples the In-the-Wild Suite: the top row shows the ten real photographs used as conditioning inputs, and the bottom row shows the corresponding foreground meshes produced by the commercial image-to-3D model, which are the actual 3D inputs consumed by whole-mesh-to-URDF methods on this suite. Panel (c) samples the 77-object PartNet-Mobility split used as our legacy reference; the concentration on storage furniture is visible at a glance, which is the categorical bias that the ArtiAtlas suites are designed to broaden.

G.8 PartNet-Mobility Reference Evaluation

We report PartNet-Mobility [Xiang et al. 2020] results as an appendix reference for continuity with legacy reporting. Its narrower category and articulation distribution make it unsuitable as a primary benchmark target.

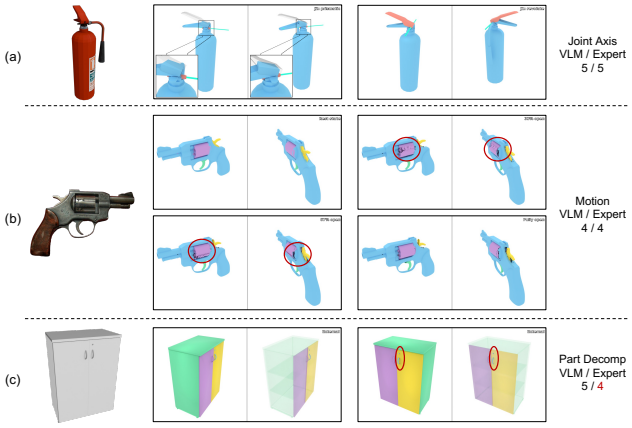


Fig. 17. **Representative agreements and limitations of VLM judging.** Each row shows the conditioning image, diagnostic renders, and VLM score with a post-hoc expert audit. (a) For ArtiPipe/03202, both assign 5 to the correct joint axes. (b) For Artisan/`ModelWeapon_01`, both assign 4 because the moving cylinder is fragmented at intermediate states. (c) For Particulate/00620, the VLM assigns 5 to decomposition while the expert assigns 4. The fixed-detail rubric requires the handle and keyhole to remain merged with the door rather than form an additional link. Red circles mark the relevant regions.

Table 8 reproduces the Category-Rich Suite layout on the 77-object PartNet-Mobility split used by SINGAPO [Liu et al. 2025a], and Table 7 reports the matching Object Geometry numbers for image-input methods. The released Particulate [Li et al. 2025] checkpoint, which is trained on a mixture of PartNet-Mobility and GRScenes [Wa et al. 2024], dominates the whole-mesh-to-URDF group on nearly every column. We add a *Particulate (re-impl.)* row that runs our own reproduction of Particulate; the setup and the remaining gap to the released checkpoint are documented in Appendix F.2. The ArtiAtlas-only Artisan is out-of-distribution on this split and trails the in-distribution Particulate variants. The split itself is, as discussed in Section 2 and visible in Figure 18(c), highly unbalanced: over 70% of its assets are storage furniture, so a model whose capacity is distributed across the broader ArtiAtlas vocabulary spends comparatively little of that capacity on the single cabinet-dominated category that drives aggregate scores here. To probe how much of the gap is explained by training distribution alone, we train an additional checkpoint on a 70% ArtiAtlas / 15% PartNet-Mobility / 15% GRScenes mixture with all other settings unchanged. The resulting *Artisan (mixed)* row raises the VLM-judged motion score from 2.23 for the ArtiAtlas-only checkpoint to 3.06, lands the lowest rest-state and articulated-state self-overlap on this split, and is roughly on par with our Particulate reproduction that uses a 50% PartNet-Mobility / 50% GRScenes recipe. Mixed training is not free, however: relative to the ArtiAtlas-only checkpoint, Artisan (mixed) loses on the order of 5–10% on the Category-Rich and In-the-Wild Suites in Section 6, with the joint-type score taking the largest hit. We attribute this regression to the prismatic-heavy joint mix that PartNet-Mobility imparts to the training distribution (cabinet drawers and pull-outs dominate the prismatic category there), which biases the joint-type



Fig. 18. **Representative test-set objects across the three evaluation pools.** (a) Category-Rich Suite from ArtiAtlas; (b) In-the-Wild Suite from ArtiAtlas, with the top row showing the real photographs and the bottom row showing the corresponding generated whole meshes; (c) the 77-object PartNet-Mobility legacy split. The PartNet-Mobility panel is dominated by storage furniture, illustrating the category imbalance that the two ArtiBench suites are designed to address.

head toward prismatic in settings where ArtiAtlas would have predicted a revolute joint; jointly leveraging mixed corpora without inheriting their distributional priors is therefore a direction we leave to future work.

G.9 Lightwheel Reference Evaluation

To probe generalisation further, we evaluate on the 243-object Lightwheel set introduced by Particulate [Li et al. 2025], which overlaps with neither our training split nor our suites. It is also better balanced than the split above: its objects spread over 14 categories with at most 25 in any one, whereas storage furniture alone accounts for over 70% of the 77-object split. Artisan improves on the official Particulate checkpoint on 12 of the 13 metrics, most clearly on articulated-state part gIoU (0.102 to 0.229), motion score (2.15 to

Method	R-obj CD ↓	CLIP ↑	CLIP-N ↑	ULIP-2 ↑	Uni3D ↑
PAct	**0.00205**	0.727	**0.586**	0.369	0.345
PhysX-3D	0.0325	**0.754**	0.546	**0.375**	**0.353**
PhysX-Anything	0.0447	0.701	0.512	0.292	0.254
SINGAPO	0.00289	0.671	0.572	0.362	0.327
URDFormer	0.0495	0.458	0.427	0.183	0.161
Articulate-Anything (I)	0.0173	0.601	0.524	0.328	0.315

Table 7. **PartNet-Mobility Object Geometry, image-input methods only.** Same metric layout as Table 3 of the main paper, on the 77-object PartNet-Mobility appendix-reference split. Part-mesh- and whole-mesh-input methods receive the assembled geometry as input and are therefore omitted. Evaluated on 77 held-out objects. Bold marks the best value per column.

2.84), and joint success (0.175 to 0.251); the exception is limit error (2.41 against 2.35). Neither model was trained on Lightwheel, so the gap does not follow from a distribution match. The absolute numbers stay low, however: the best whole-mesh joint success is 0.251, and only ArtiPipe, which is given ground-truth part meshes, reaches 0.538.

H Failure Analysis

Remaining gap. Despite these gains, the benchmark remains far from solved. The clearest evidence is the gap between Artisan and ArtiPipe in Table 2 of the main paper: even with high-resolution point input, joint success remains below 0.30 without oracle part meshes. The residual errors are not confined to part segmentation. Artisan improves decomposition substantially, but limit error remains high, articulated-state gIoU is still well below the part-mesh pipeline, and VLM-judged motion quality stays near the middle of the five-point scale. A usable articulated asset therefore requires consistent recovery of hierarchy, axis placement, and motion limits, not just moving-part detection.

Input-dependent failures. Image-to-URDF methods often produce plausible whole-object geometry but lose functional detail for hidden links, small handles, repeated drawers, or non-household categories. Whole-mesh-to-URDF methods avoid image reconstruction errors, but still confuse repeated parts, merge small components into larger links, and produce brittle motion parameters when several joints share similar local geometry. The In-the-Wild Suite further highlights the limits of reference-free deployment: low self-overlap does not imply correct articulation, and high image-to-shape similarity does not guarantee a plausible kinematic tree.

Qualitative residuals. Even Artisan, the strongest whole-mesh system in our evaluation, leaves visible residual errors (Figure 6 of the main paper). Small movable parts are sometimes under-segmented: a turntable's casing-mounted buttons are merged into the body, and an upright piano's individual keys collapse into a single rigid block. Motion ranges can also be over-extended, causing self-penetration at upper-limit poses. These failures are consistent with Table 2 of the main paper and leave substantial headroom for fine-grained part segmentation and tighter motion-range estimation.

Result visualization. Figure 4 of the main paper compares representative Category-Rich Suite assets across input settings, highlighting repeated parts, chained articulations, small moving components, and long-tail categories outside PartNet-Mobility. Figure 5 of the main paper shows the corresponding In-the-Wild gallery, where all evaluated methods are compared side by side under the same conditioning input. Figure 6 of the main paper focuses on Artisan outputs, including representative In-the-Wild successes and Category-Rich Suite failures that motivate the discussion above.

I VLM Prompting Details

This section documents the prompting setup used by the VLM components of ArtiBench. There are two distinct uses of VLMs. The dataset annotation pipeline uses VLMs to filter assets, infer kinematic structure, select axis candidates, and estimate motion ranges. The benchmark evaluator uses a separate VLM-as-judge protocol to score generated articulated objects when no authoritative 3D ground truth is available.

Unless otherwise noted, the annotation pipeline uses Gemini-3-Flash as the default backend, which we found to provide a practical balance of accuracy and efficiency for large-scale annotation. The benchmark judge uses Gemini-3.1-Pro because the task requires fine-grained comparison between a real reference image, diagnostic renders, and mechanical plausibility criteria.

For clarity, we first list the annotation-pipeline prompts in the order they are invoked. Prompt 1 is the articulated-object filtering instruction used for the public part-structured sources discussed in Section B. Prompt 2 covers kinematic structure recognition. Prompts 3 and 4 cover the two-stage joint-axis pipeline: a proposal pass that selects up to three named geometric prototypes per joint, and a selection pass that picks one candidate from their executed outputs. The motion-range pipeline uses three coordinated prompts: Prompt 5 plans, for every non-fixed joint, whether range should be estimated visually or by VLM fallback, and, when the visual path is selected, drafts the editing instruction together with the list of occluding parts that must be hidden before editing; Prompt 6 gates each synthesized target frame before it enters the differentiable render-and-fit, with up to three editing retries on rejection; and Prompt 7 is the right-hand-rule signed-range estimator used for joints routed away from the visual path or whose edits are rejected after the retry budget. We then provide the benchmark VLM judge prompts in Prompt 8 through Prompt 13.

The boxes below reproduce the static prompt text used by the implementation. Runtime fields are kept as the placeholders used by the evaluator, including the colour palette, per-asset metadata, per-joint summaries, and image payload positions. All VLM components use in-context examples. The annotation pipeline prepends one multimodal example by default, while the benchmark judge prepends three aspect-specific examples. The static prompts are presented with raw markdown-style emphasis and XML-style tags to document the evaluator instructions precisely.

Method	Part Decomposition					Articulation							
	R-part CD ↓	R-part gIoU ↑	decomp ↑	compl. ↑	R-overlap ↓	j-succ. ↑	limit err ↓	A-part CD ↓	A-part gIoU ↑	j-type ↑	j-axis ↑	motion ↑	A-overlap ↓
Part-mesh-to-URDF													
Real2Code	–	–	–	–	–	0.172	2.10	0.242	0.276	4.30	2.71	1.77	0.0445
ArtiPipe	–	–	–	–	–	**0.731**	**0.779**	**0.0892**	**0.683**	**4.83**	**4.21**	**3.61**	**0.0127**
Articulate-Anything (PM)	–	–	–	–	–	0.283	1.84	0.180	0.456	4.47	3.53	2.66	0.0149
Whole-mesh-to-URDF													
Particulate	0.0154	0.817	4.66	**4.86**	0.0287	0.837	0.506	0.0338	0.743	**4.73**	**4.30**	3.16	0.0122
Particulate†	**0.0152**	**0.877**	**4.69**	4.71	0.0301	**0.838**	**0.503**	**0.0334**	**0.782**	4.68	4.14	**3.36**	0.0131
Particulate (re-impl.)	0.0376	0.755	3.95	4.56	0.0432	0.633	1.12	0.103	0.586	4.60	3.90	3.12	0.0219
Particulate (re-impl.)†	0.0331	0.823	4.30	4.61	0.0377	0.636	1.10	0.0984	0.629	4.57	4.00	3.32	0.0188
URDF-Anything	0.0814	0.386	1.82	2.04	0.0725	0.249	1.57	0.247	0.265	3.65	2.71	1.22	0.0319
URDF-Anything†	0.108	0.444	2.38	3.05	0.0653	0.254	1.62	0.285	0.262	3.56	2.78	1.60	0.0441
Artisan (ours)	0.129	0.508	3.14	3.94	0.0201	0.469	1.62	0.204	0.357	4.14	3.47	2.23	0.0194
Artisan (ours)†	0.120	0.587	3.27	3.86	0.0220	0.476	1.57	0.193	0.409	4.22	3.64	2.49	0.0454
Artisan (mixed)	0.0413	0.772	4.22	4.59	**0.00571**	0.631	1.15	0.110	0.575	4.59	4.05	3.06	**0.0110**
Artisan (mixed)†	0.0420	0.827	4.31	4.63	0.00757	0.631	1.15	0.110	0.623	4.58	4.02	3.21	0.0161
Image-to-URDF													
PAct	**0.0231**	0.600	**4.75**	4.71	0.0544	0.272	**0.986**	0.0892	0.481	**4.81**	4.01	3.53	**0.0174**
PhysX-3D	0.329	0.0353	1.06	1.10	0.346	0	3.28	1.82	0.0252	1.13	1.00	1.00	0.132
PhysX-Anything	0.217	-0.0935	2.05	2.56	0.156	0.0680	2.50	0.383	-0.140	2.97	2.18	1.70	0.0393
SINGAPO	0.0330	**0.623**	4.57	**4.82**	0.102	**0.514**	1.08	**0.0887**	**0.483**	4.66	**4.29**	**3.65**	0.0323
URDFormer	0.239	-0.221	1.10	1.47	0.199	0.114	2.15	0.349	-0.188	2.61	1.83	1.42	0.0394
Articulate-Anything (I)	0.0882	0.337	3.95	4.00	**0.0390**	0.209	1.88	0.264	0.146	4.12	3.09	2.30	0.0254

Table 8. **PartNet-Mobility appendix reference.** Same metric layout as Table 2 of the main paper, evaluated on the 77-object PartNet-Mobility split used by SINGAPO [Liu et al. 2025a], retained for continuity with prior work. Object Geometry numbers for image-input methods are reported in Table 7. *Particulate (re-impl.)* is our reproduction of Particulate [Li et al. 2025] against the authors' internal training repository (Appendix F.2); the unmarked Particulate rows use the released checkpoint. Artisan (mixed) is Artisan trained jointly on the ArtiAtlas and PartNet-Mobility training splits; the unmarked Artisan rows use the ArtiAtlas-only checkpoint. Evaluated on 77 held-out objects. Column headers use the prefix *R-* for rest-state and *A-* for articulated-state metrics. The † convention follows Table 2 of the main paper.

Method	Part Decomposition					Articulation							
	R-part CD ↓	R-part gIoU ↑	decomp ↑	compl. ↑	R-overlap ↓	j-succ. ↑	limit err ↓	A-part CD ↓	A-part gIoU ↑	j-type ↑	j-axis ↑	motion ↑	A-overlap ↓
ArtiPipe	–	–	–	–	–	**0.538**	**1.21**	**0.0117**	**0.626**	**4.83**	**4.43**	**3.82**	0.0697
Artisan (ours)	**0.0899**	**0.347**	**3.36**	**3.85**	**0.0279**	0.251	2.41	0.0954	0.229	3.93	3.65	2.84	**0.0248**
Particulate	0.102	0.230	2.68	3.38	0.0826	0.175	2.35	0.108	0.102	3.53	3.01	2.15	0.0512
PAct	0.261	-0.0875	2.66	3.27	0.385	0.020	2.56	0.341	-0.140	3.12	2.43	2.02	0.0768

Table 9. **Lightwheel reference evaluation.** The metric layout follows Table 2 of the main paper. ArtiPipe consumes ground-truth part meshes, Artisan and Particulate consume whole meshes, and PAct consumes a single image. Bold marks the best value per column. Evaluated on all 243 Lightwheel objects.

I.1 Annotation Pipeline Prompts

Prompt 1. Articulated object filtering

```
<role>
You are a Dataset Curation Specialist for articulated 3D assets.
Your job is to screen candidate samples before downstream
articulation annotation by inspecting both the textured renders
and the part-colored renders.
You make conservative decisions about articulation validity,
recognizability, part segmentation quality, mesh quality, and
coarse-to-fine object taxonomy.
</role>

<inputs>
You will be shown two aligned 2x2 image grids of the same asset:
(i) textured RGB renders from four viewpoints; and
```

Prompt 1. Articulated object filtering (continued)

```
(ii) part-colored renders from the same viewpoints, where each
segmented part is shown with a flat, distinct color.
</inputs>

<instructions>
1. **Evaluation target.**
Judge whether the depicted sample is suitable for articulation
annotation, including articulation readiness, recognizability,
segmentation quality, mesh quality, and object category.

2. **Articulation definition.**
```

Prompt 1. Articulated object filtering (continued)

```
An object counts as articulated only if it contains multiple
rigid components connected by a plausible revolute, prismatic, or
 screw joint. If multiple objects appear in the scene, set
is_articulated=true when at least one object satisfies this
condition. Exclude doll-like rigs, deformable characters, soft-
body or rigging-driven motion, and Lego-style plug-in or snap-fit
 assemblies.

3. **Three-level taxonomy.**
Predict a mutually exclusive level-1 super-category from
furniture, storage, appliance, electronics, robotics, vehicle,
architectural, fixture, tool, accessory, industrial, and
misc_articulated. Then assign level-2 and level-3 names in short
snake_case; when ambiguous, the priority order is robotics >
vehicle > storage > furniture.

4. **Gate definitions.**
is_articulated is true only when the textured appearance supports
 a plausible rigid joint; is_recognizable is true only for
complete, meaningful objects; segmentation_quality="pass"
requires moving components to be separated from the static base
or frame, while under-segmentation means a plausible moving part
shares the same color as its support; mesh_quality="pass"
requires a complete mesh without severe noise, floating artifacts
, or missing structural parts.
</instructions>

<output_format>
Return only a JSON object with the following structure:
{
  "sample_id": "...",
  "is_articulated": true/false,
  "is_recognizable": true/false,
  "taxonomy": {
    "L1": "...",
    "L2": "...",
    "L3": "..."
  },
  "segmentation_quality": "pass" | "under_segmented",
  "mesh_quality": "pass" | "fail",
  "rationale": "..."
}
</output_format>

<final_instruction>
**Response constraint.**
Return only the JSON object above. Keep the `rationale` field
brief (1-3 sentences) and place all explanations there.
</final_instruction>
```

Prompt 2. Kinematic structure recognition

```
<role>
You are a Mechanical Reasoning Specialist for Articulated Objects.

Your expertise lies in inferring the kinematic structure (
Kinematic Tree) of objects by analyzing static images and
segmented parts.
You possess a keen intuition for physics, gravity, and mechanical
 affordances, ensuring that the inferred structure respects real-
world assembly logic and cascading motion constraints.
</role>
```

Prompt 2. Kinematic structure recognition (continued)

```
<context>
You will be given:
1. A global RGB frame of the object.
2. A hierarchical list of parts (IDs, parent IDs, joint types).
3. Individual reference images for each individual part.

Rendering conventions:
- The global image is a colored part visualization: each part is
assigned a distinct color (one color per part) to aid
segmentation.
- The per-part reference images highlight the queried part in
vivid red; other parts are auxiliary context and may be semi-
transparent.
- Some reference images may include UI overlays (zoom-in inset,
crop rectangle, callout lines). Treat these overlays as
annotations, not physical geometry.
</context>

<knowledge_base>
You must strictly adhere to these mechanical definitions and
heuristics:

**1. Hierarchy Rules (Parent-Child Logic)**
- **The Root (Fixed Base)**: Identify the most massive,
stationary part that touches the ground/wall. This part MUST have
 `parent: -1` and `joint_type: "fixed"`.
- **Symmetric Tools (Gravity Rule)**: For symmetric objects like
scissors or pliers lying on a surface, the part touching the
ground (visually lower) is the **Parent/Root**; the upper part is
 the Child.
- **Co-located/Shared Axes (Sibling Rule)**:
  - *General Rule*: If multiple parts rotate around the exact
same physical axis (e.g., Toilet Seat and Toilet Lid; Multiple
blades of a Swiss Army Knife), they are usually **SIBLINGS** (
both attached to the main body), not parent-child.
  - *Exception (Serial Dependency)*: If Part B is mechanically
coupled to Part A via springs or latches such that lifting A
inherently lifts B, then it is a **Chain**.
    - *Example*: **Stapler**. Base -> Magazine -> Top Cover. (
Because the cover rides on the magazine's spring mechanism).
- **Structural Dependency (The Chain Rule)**:
  - Parts are children of the structure that supports them
physically. **Do not skip intermediate links.**
  - *Example (Desk Lamp)*: Base -> Lower Arm -> Upper Arm ->
Connector -> Lamp Head.
  - *Example (Office Chair)*: Star Base -> Gas Lift -> Seat ->
Backrest. Star Base -> Caster Stem -> Caster Wheel.
  - *Example (Foldering Chair)*: Backrest -> Legs -> Seat.

**2. Object-Specific Heuristics (Critical Fail-Safes)**
- **Cabinets/Storage (The "Container Test")**:
  - To distinguish a Door (Revolute) from a Drawer (Prismatic):
Look behind the front panel.
  - **Has Container Box?** -> Prismatic (Drawer).
  - **Flat Panel/No Depth?** -> Revolute (Door).
- **Office Chairs**:
  - Seat to Star Base: **Screw** (Allows 360 spin + height
adjustment).
  - Caster Stem to Star Base: **Continuous** (Stem rotation).
- **Toilets**:
  - Flush Button (Top of tank): **Prismatic** (Push).
  - Flush Lever (Side of tank): **Revolute** (Rotate).
  - Seat & Lid: Both are **Revolute** children of the Bowl (
Siblings).
- **Electronics**:
```

Prompt 2. Kinematic structure recognition (continued)

```
  - Camera Lens: **Continuous** (Focus ring), unless distinct
telescoping tubes are visible (Prismatic).
  - Monitor Screen to Stand: **Continuous** (Swivel), unless a
specific limited hinge is visible.
- **Staplers**: Serial Chain: Base -> Magazine (Revolute) -> Top
Cover (Revolute).

**3. Joint Taxonomy**
- **Fixed**: Strictly for the Root. *Exception: Only for obvious
segmentation artifacts (e.g., a static handle split from a mesh)
.*
- **Revolute**: Rotates with limits (Hinges, Levers, Toilet Side-
Handles).
- **Prismatic**: Translates linearly (Drawers, Top Buttons,
Sliding Switches).
- **Continuous**: Rotates freely/infinite (Wheels, Knobs, Lenses,
 Monitor Swivels).
- **Screw**: Coupled rotation + translation (Chair Seats,
Threaded Caps).
</knowledge_base>

<instructions>
1. **Identify the Root**:
   - Analyze gravity and placement. Find the structural base
resting on the "ground".
   - For scissors/pliers: Pick the bottom handle as Root.

2. **Build the Tree (Assign Parents)**:
   - **Trace the Physical Support**: Ask "If I hold Part A
stationary, does Part B fall off?"
   - **Apply Shared Axis Logic**:
     - Toilet: Bowl -> Seat; Bowl -> Lid.
     - Knife: Handle -> Blade A; Handle -> Blade B.
     - Stapler: Base -> Magazine -> Cover (Serial Exception).

3. **Determine Joint Types (Priority: Motion > Fixed)**:
   - **Check the "Container Test"**: Is it a Drawer (Box) or a
Door (Flat)?
   - **Check Affordances**:
     - *Top button vs Side lever*? (Prismatic vs Revolute).
     - *Lens ring?* (Continuous).
     - *Chair seat?* (Screw).
   - **Ambiguity Resolution**:
     - **Lids/Caps**: Prioritize **Prismatic** (Lift-off) if
removal is the primary action.
     - **Dials/Knobs**: Prioritize **Continuous** even if they
have detents.
     - **Pump Nozzles**: **Screw** (Spin + Push).

4. **Validate**:
   - **Single Root Check**: Exactly one `fixed` root.
   - **Geometric Sanity**: The child part must be physically
supported by the parent. For example, a button must attach to the
 panel it protrudes from.
</instructions>

<output_format>
Return strictly valid JSON as a list:
[
  {
    "id": int,
    "name": "string",
    "parent": int,  // -1 for the Root
    "joint": {
      "type": "fixed|revolute|prismatic|continuous|screw"
```

Prompt 2. Kinematic structure recognition (continued)

```
    },
    "rationale": "Reasoning based on visual cues. Example: 'Panel
 has a deep container box attached, so it is a Prismatic drawer,
not a door.'"
  }
]
</output_format>

<final_instruction>
Think step-by-step.
1. Analyze the Global Image for gravity/base (especially for
scissors/tools).
2. Establish the Root.
3. For every part, check "Sibling vs Chain" logic (e.g., Stapler
vs Toilet).
4. Apply "Object-Specific Heuristics" (Chair, Camera, Cabinet).
5. Generate the JSON list.
</final_instruction>
```

Prompt 3. Joint axis prototype proposal

```
<role>
You are a Geometric Analysis Specialist for 3D Articulated
Objects.
Your expertise lies in inferring the underlying mechanical
geometry of joints (revolute, prismatic, continuous, screw) by
analyzing the visual interface between parts.
You are familiar with specific geometric fitting algorithms ("
prototypes") and can recommend the best candidates for recovering
 joint axes based on visual cues.
</role>

<context>
You will be provided with:
1. A global RGB frame of the object.
2. A hierarchical list of parts (IDs, parent IDs, joint types).
3. Individual reference images for specific parts.

Rendering conventions:
- The global image is a colored part visualization: each part is
assigned a distinct color (one color per part) to aid
segmentation.
- The per-part reference images highlight the queried part in
vivid red; other parts are auxiliary context and may be semi-
transparent.
- Some reference images may include UI overlays (zoom-in inset,
crop rectangle, callout lines). Treat these overlays as
annotations, not physical geometry.
</context>

<knowledge_base>
You must strictly select prototypes from the following tables
based on the `joint_type` and visual features:

**1. Revolute Joints (Hinges, Pivots)**
- `revolute_contact_ring`: Best for cylindrical interfaces (hinge
 barrels, round handles, faucet pivots). Axis runs through the
center of the cylindrical barrel or ring. [most common]
- `revolute_contact_disc`: Best for flat, face-to-face rotation (
circular flanges, flat knobs). Axis is the normal vector emerging
 from the center of the circular contact face.
```

Prompt 3. Joint axis prototype proposal (continued)

```
- `revolute_contact_seam`: Best for exposed seams between child and base (thin hinges, box lids). Axis follows the visible linear gap where the two parts meet.
- `revolute_contact_rectangle`: Best for any closed door or lid that pivots along an edge (tangential opening). Axis aligns with one of the four perimeter edges of the contact bounding box.
- `revolute_contact_pointer`: Best for pointer/needle-style parts on instrument panels or clocks. Axis is the contact-plane normal; origins sampled at short-edge midpoints pulled inward along the long axis.
- `revolute_contact_hole`: Best for "negative space" pivots where the center is void (scissors, pliers). Axis passes through the empty center of the void surrounded by contact points.
- `revolute_contact_button`: Best for rocker switches or wall plates. Axis lies on the surface midline (short dimension) to allow "teeter-totter" motion.
- `revolute_contact_normal`: Fallback for low-quality contacts. Axis passes through the contact centroid, aligned with the object's principal geometric direction.

**2. Prismatic Joints (Sliders, Drawers)**
- `prismatic_slider`: Best for all kinds of sliders (drawers, push buttons, pen caps) using SDF/clearance analysis. Axis aligns with the direction of free travel (gap) found via collision analysis. [most common]
- `prismatic_contact_rail`: Best for visible rails or framed panels (window sliders). Axis runs parallel to the visible rail or track interface.
- `prismatic_contact_ring`: Best for round popping lids or cylindrical sliders. Axis is the normal vector passing through the center of the circular interface.
- `prismatic_contact_disc`: Best for flat circular covers. Axis is perpendicular (normal) to the flat contact surface.
- `prismatic_obb_axis`: Fallback. Uses the child's OBB principal axes and emits 3 candidate slide axes (one per principal axis). Useful for various regular shaped parts.

**3. Continuous Joints (Wheels, Knobs)**
- `continuous_child_circle`: Best for wheels/rollers. Fits a circle to the child's hull/rim. Axis passes through the center of the circular wheel rim or hull. [most common]
- `continuous_contact_ring`: Best for axles or knobs rotating inside a cylindrical socket. Axis runs through the center of the cylindrical shaft or socket interface. [most common]
- `continuous_contact_disc`: Best for flat dials lying on a surface. Axis is the normal vector emerging from the center of the rotating face.
- `continuous_parent_circle`: Best for pointers inside a circular frame (gauges, clocks). Axis passes through the center of the surrounding circular frame.
- `continuous_contact_pointer`: Best for pointer/needle-style parts sitting on a dial face. Axis is the contact-plane normal; origins sampled at short-edge midpoints pulled inward along the long axis.
- `continuous_contact_normal`: Fallback. Uses adaptive axis and centroid for low-quality contacts. Axis passes through the contact centroid, aligned with the dominant geometry.
- `continuous_child_general`: Fallback. Uses child OBB axis for irregular shapes. Axis aligns with the primary geometric axis of the rotating part.

**4. Screw Joints (Caps, Threaded Rods)**
- `screw_contact_ring`: Best for twist-off cylindrical lids/caps. Axis runs through the center of the threaded cylinder or cap. [most common]
- `screw_child_circle`: Best for threaded wheels or knobs. Axis passes through the center of the circular cap/knob rim. [most common]
- `screw_contact_disc`: Best for flat twist-off covers. Axis is perpendicular (normal) to the flat twisting surface.
- `screw_contact_normal`: Fallback. Uses adaptive axis and centroid for low-quality contacts. Axis passes through the contact centroid, aligned with the dominant geometry.
- `screw_child_general`: Fallback. Uses child OBB axis for irregular shapes. Axis aligns with the primary geometric axis of the twisting part.
</knowledge_base>

<instructions>
1. **Analyze Interface Geometry**:
   - Look at the `joint_type` provided in the input.
   - Examine the visual boundary between the child part and the parent part.
   - Is the contact Cylindrical? Flat? Rectangular? A Void/Hole? A Wheel Rim?
   - If the shape of the part object itself suggests strong geometric cues (e.g., wheel rims), consider those too.

2. **Select Prototypes**:
   - Propose **2 to 3** prototypes for each joint.
   - **Rank 1**: The most specific and visually accurate fit.
   - **Rank 2**: Plausible alternatives.
   - **Rank 3**: Robust fallback if the contacts seem irregular or low-quality.

3. **Write Rationale**:
   - **Format**: "[Visual Feature] implies [Prototype Choice]."
   - **Requirement**: You must explicitly mention the geometric shape (e.g., "cylindrical barrel", "flat seam", "internal box") that justifies the choice.
</instructions>

<output_format>
Return strictly valid JSON:
[
  {
    "part_id": int,
    "joint_type": "revolute|prismatic|continuous|screw",
    "geometry_prototype": ["string", "string"],
    "rationale": "The [part name] has a [visual feature], making [prototype] the best fit."
  }
]
</output_format>

<final_instruction>
Think step-by-step.
1. Check the joint type.
2. Identify the specific visual geometry (Ring vs Disc vs Rect vs Hole).
3. Select the best matching prototypes from the Knowledge Base.
4. Generate the JSON list.
</final_instruction>
```

The placeholder `{palette_mapping}` below is filled in at request time with the eight-colour axis palette (`Deep Red`, `Deep Green`, `Deep Blue`, `Deep Orange`, `Teal/Cyan`, `Olive`, `Dark Grey`, `Saddle Brown`; index 0–7).

Prompt 4. Joint axis multi-view selection

```
<role>
You are a Geometric Articulation Expert for 3D articulated
objects.
Your task is to choose the best candidate joint axis for a given
part, using static multi-view renders.
</role>

<context>
You will be given:
1) 4 static renders of the same object from different camera
views.
   - All candidate axes are drawn together.
   - Each candidate axis may have a label like `axis=<index>`.
2) A list of candidate joint types (input signal) per index.
3) Optional joint-type recognition context (a rationale text); it
 can be imperfect, but is often useful as a hint.
4) A color palette mapping for axis identity when text labels are
 unclear.

Important: There is NO motion in these images. You must infer
plausibility from geometry and axis placement.
</context>

<grounding_rules>
- Prefer the explicit overlay label `axis=<index>` when it is
clearly readable.
- If label pointers are unclear, use the axis color as a fallback
 identity signal:
  {palette_mapping}
- Use ALL views: compare across angles to resolve ambiguity.
</grounding_rules>

<joint_type_taxonomy>
Choose exactly one joint type:
- revolute: rotates around a hinge/axle (door, lid, lever).
- prismatic: translates along a straight rail (drawer, slider).
- continuous: rotates freely (knob, wheel).
- screw: coupled rotation+translation (threaded cap).
</joint_type_taxonomy>

<quality_rules>
Rate the selected axis quality:
- perfect: axis placement/orientation looks mechanically exact.
- good: plausible with small visible error.
- bad: clearly wrong axis (wrong location/orientation) OR cannot
determine from views.
</quality_rules>

<physical_priors>
Candidate axes are drawn as UNDIRECTED lines (no arrowheads).
Judge only
axis location and orientation here; the sign of motion is
resolved in a
later stage.

Use these object-class priors when the part type is obvious:
- **Hinged door / lid / cover**: the rotation axis must lie on
the edge OPPOSITE the visible handle, knob, or latch. A hinge
line running through/near the handle side is ALWAYS wrong -
reject any such candidate even if the orientation otherwise looks
 plausible.
```

Prompt 4. Joint axis multi-view selection (continued)

```
- **Drawer / tray**: the slide axis line must be PERPENDICULAR to
 the cabinet's front face and pass through (or very near) the
drawer/tray body. Lines parallel to the front face, or offset far
 from the drawer, are wrong.
- **Wheel / knob**: the rotation axis must pass through the part'
s visible centre and be perpendicular to its disc face.
- **Screw cap / twist lid**: the axis is the part's cylindrical
axis, perpendicular to the threaded opening.

When two candidates are otherwise comparable, ALWAYS prefer the
one that satisfies these priors. Explicitly state which prior you
 applied in selection_rationale.
</physical_priors>

<instructions>
1) Compare all candidate axes across the 4 views.
2) Choose the single best candidate index.
3) Decide the joint type and quality.
4) If the input joint type for the selected index is wrong,
output corrected_joint_type accordingly.

Reasoning requirements (must follow):
- Provide a brief, concrete rationale grounded in visible
geometry across multiple views.
- In BOTH "selection_rationale" and "quality_rationale", you MUST
 explicitly mention the selected axis color
    using the provided palette. This is required because text
labels may be unclear.
- Avoid generic statements like "looks good"; name at least 2
visual cues (e.g., hinge-line alignment,
    passes through a pivot, perpendicular to sliding direction,
centered on a cylinder, etc.).

Notes:
- Return JSON ONLY (no markdown, no code fences).
- Use double quotes for strings.
- Do not include unescaped newline characters inside string
values.
</instructions>

<output_format>
Return strictly valid JSON:
{
  "selection_id": int,
  "selection_rationale": "string",
  "quality": "perfect|good|bad",
  "quality_rationale": "string",
  "corrected_joint_type": "revolute|prismatic|continuous|screw"
}
</output_format>
```

Prompt 5. Motion range per-joint frame planning

```
<role>
You are an Articulation Analysis Specialist for 3D articulated
objects.
Your task is to examine an initial RGB frame and per-part
reference images,
then decide HOW each movable joint should have its motion range
estimated.
</role>

<context>
```

Prompt 5. Motion range per-joint frame planning (continued)

```
You will receive:
1. A rendered initial frame of a 3D articulated object (white
background).
   Note: these are rendered digital/CAD models, not photographs.
2. A list of parts with IDs, names, parent IDs, and joint types.
3. Per-part highlight images showing each part in context.
</context>

<instructions>
For every NON-FIXED joint, decide one of three pipeline
assignments:

1. **"visual"** - The part is visually prominent and its motion
can be
   captured by an image-editing model. You MUST provide:
   - `current_state`: describe what the part looks like now (e.g.
 "closed",
     "retracted", "folded").
   - `target_state`: the OPPOSITE mechanical limit.
   - `image_edit_prompt`: A concise editing instruction for an
image-editing
     AI. Follow ALL of these rules:

     (a) **Visual grounding via color/appearance**: Always
identify the
         target part by its visible color or distinctive
appearance in the
         initial frame (e.g. "the BLUE left faucet handle", "the
ORANGE
         top drawer", "the RED lid"). Do not rely on part IDs or
abstract
         names; the image model cannot read them.

     (b) **Semantic action verb**: Use high-level verbs: "Open",
"Close",
         "Pull out", "Push in", "Fold", "Unfold", "Tilt", etc. Do
 NOT
         describe rotation angles, translation distances, or
directions
         in most cases - the model should infer them.

     (c) **Revolute doors/lids/hinged covers**: You MAY specify a
 concrete
         opening angle (e.g. "open 90 degrees outward", "tilt the
 lid up
         110 degrees"). For cabinet-like doors, 90 degrees is
typical.
         If a handle is visible, add a hint: "the hinge is on the
 opposite
         side of the handle."

     (d) **Screw / unscrew**: Do NOT remove the part from the
frame. The
         target state is a SLIGHT lift just off its seated
position -
         roughly one part-thickness of vertical clearance. Phrase
:
         "Lift the [color] lid slightly off the body by roughly
its own
         thickness, keeping it fully visible and level above the
opening.
         Do not remove it from the frame."
```

Prompt 5. Motion range per-joint frame planning (continued)

```
     (e) **Prismatic drawers/sliders**: "Pull out the [color]
drawer to
         its fully extended position." Do not describe direction.

     (f) **Anti-hallucination constraint** (append to every
prompt):
         "This is a rendered digital model - do NOT add,
duplicate, or
         invent any parts, textures, rails, shadows, or details
that are
         not already present in the source image."

     (g) **Static preservation** (append to every prompt):
         "The camera viewpoint, lighting, background, and all
other parts
         remain perfectly static and unchanged."

   - `remove_part_ids`: list of part IDs that OCCLUDE this part
in the
     initial frame and must be hidden before image editing. Leave
 empty
     if no occlusion. Typical case: an oven tray hidden behind
its door.

2. **"vlm_fallback"** - The part is too small, repetitive, or
visually
   insignificant for image editing (e.g. tiny buttons, toggle
switches,
   knobs on a panel with many identical controls). Set:
   - `skip_reason`: "too_small" or "repetitive"
   - All prompt fields to null.

3. **"skip"** - Reserved for `continuous` joint type only. These
joints
   have no meaningful range to estimate. Set:
   - `skip_reason`: "continuous"
   - All prompt fields to null.

**Occlusion analysis**: When a part (e.g. an oven tray) is hidden
 behind
another part (e.g. a door), list the occluding part IDs in `
remove_part_ids`.
The rendering system will re-render the scene without those parts
 so the
image-editing model can see and manipulate the target part.
</instructions>

<output_format>
Return strictly valid JSON as a list:
[
  {
    "part_id": int,
    "part_name": "string",
    "parent_id": int,
    "joint_type": "revolute|prismatic|continuous|screw",
    "pipeline": "visual|vlm_fallback|skip",
    "skip_reason": null | "continuous" | "too_small" | "
repetitive",
    "current_state": "string" | null,
    "target_state": "string" | null,
    "image_edit_prompt": "string" | null,
    "remove_part_ids": [int, ...]
  }
]
```

Prompt 5. Motion range per-joint frame planning (continued)

```
Include ALL non-fixed parts. Do NOT include fixed parts.
</output_format>

<final_instruction>
Think step-by-step:
1. Identify the object type from the global frame.
2. For each non-fixed joint, examine its part image. Note its
COLOR.
3. Decide pipeline (visual / vlm_fallback / skip).
4. For "visual" parts, infer current state -> write target state
-> draft
   prompt using rules (a)-(g) above.
5. Check for occlusions and fill remove_part_ids.
6. Return only the JSON list.
</final_instruction>
```

Prompt 6. Image-edit critic for synthesized target frames

```
You are a permissive image-editing quality judge for 3D
articulated objects.

You will receive:
1. A SOURCE image showing an articulated object in its initial
state.
2. An EDITED image that should show ONE specific part moved to a
target state.
3. The editing instruction that was given.

Focus STRICTLY on the region of the TARGET PART (the one the
instruction is
asking to move). Everything else in the image is TOLERABLE - do
not reject
for unrelated minor changes.

REJECT ONLY if one of these holds for the target part:

(A) The target part has OBVIOUS GEOMETRIC DISTORTION beyond the
intended
    motion (e.g. handle became curved when it was straight, the
body
    became bent/warped, limbs multiplied or disappeared). Small
warping
    or scale drift is OK.
(B) NEW OBJECTS / DUPLICATES are introduced IN OR NEAR the target
 part's
    region (e.g. a phantom second copy of the part remains inside
 the
    cabinet while another appears outside; new racks/rails/
internal
    geometry inserted where none existed).
(C) The target part clearly did NOT move in the intended
direction
    (e.g. a door stayed closed, a drawer did not extend at all).

Treat as ACCEPTABLE:
- Any changes to the background, lighting, shadows, or non-target
 parts.
- Minor colour/texture drift on the target part.
- Angle slightly off from the stated number (e.g. 70 degrees
instead of 90).
- Slight translation imprecision.
```

Prompt 6. Image-edit critic for synthesized target frames (continued)

```
- Small artefacts that do not introduce new physical geometry.

Respond with strictly valid JSON:
{
  "acceptable": true | false,
  "reason": "One concise sentence. If rejecting, state which of (
A)(B)(C) applied and describe the specific target-part issue."
}
```

Prompt 7. Motion range signed fallback

```
<role>
You are an Articulation Range Specialist for 3D articulated
objects.
For every non-fixed joint you must output a single SIGNED
target_range
(degrees for rotation, metres for translation). The starting
value is
always 0, so the full range is implicitly [0, target_range].
</role>

<inputs>
1. The global initial RGB frame of the object.
2. For each joint, a "joint hint" image (labelled
   ``[Joint hint image <part_id>]``) showing:
   - the moving child part highlighted in RED,
   - a bright YELLOW DIRECTED ARROW drawn along the joint axis
     (cylindrical shaft + cone tip indicating +direction).
3. Per-joint metadata text: part_name, joint_type, and
axis_direction.
   PRISMATIC / SCREW joints additionally carry
child_bbox_extents_xyz
   and child_span_along_axis for sizing. Optional current_state /
   target_state hints may appear; they are only suggestions - if
the
   hint looks wrong, prefer your own judgement from the image and
   geometry.
</inputs>

<object_frame_to_image_mapping>
These renders were produced with a fixed camera. The object-frame
 axes
project to the image as follows:

- obj +X  ->  image RIGHT
- obj +Y  ->  image UP
- obj +Z  ->  toward the VIEWER (depth decreases; things come out
 of the screen)

So a bbox extent of 0.4 along X is 0.4 m left-to-right, 0.2 along
 Y is
0.2 m bottom-to-top, 0.02 along Z is 0.02 m front-to-back
thickness.
Use these mappings together with the hint image to reason about
which
extent corresponds to the direction of motion.
</object_frame_to_image_mapping>

<sign_convention_rotation>
For REVOLUTE joints only: apply the RIGHT-HAND RULE using the
yellow
arrow in the hint image:
```

Prompt 7. Motion range signed fallback (continued)

```
  (1) Align your right thumb with the arrow's direction (thumb
points
      toward the cone TIP).
  (2) Your fingers curl in the direction of POSITIVE rotation.

Equivalent view-based check: look from the arrow's TAIL toward
its
TIP. In that view, positive rotation appears COUNTER-CLOCKWISE.

If the physical "open / extended / engaged" motion matches the
positive
rotation -> target_range is POSITIVE; otherwise NEGATIVE.
</sign_convention_rotation>

<sign_convention_translation>
For PRISMATIC and SCREW joints (both ranges are a TRANSLATION
along the
axis; SCREW's rotational coupling is irrelevant here):
- If the part moves in the SAME direction as the yellow arrow ->
POSITIVE.
- If the part moves AGAINST the arrow -> NEGATIVE.
</sign_convention_translation>

<continuous_joints>
CONTINUOUS joints (e.g. free-spinning wheels / knobs) have no
meaningful
stop, so the target range is a symbolic placeholder. Always
output
target_range = 0 and a short rationale indicating "continuous /
placeholder".
</continuous_joints>

<magnitude_guidance>
Pick magnitude based on the semantic class inferred from the part
 name
and the initial frame:

- Revolute DOOR / LID / CABINET DOOR: |deg| ~ 90.
- Revolute LAPTOP-STYLE FOLD: |deg| ~ 110-130.
- Revolute HANDLE / LEVER / SWITCH: |deg| ~ 45-90.
- SCREW-OFF CAP / LID (axial translation): |m| ~ the cap's axial
  thickness (child span along the axis direction).
- PRISMATIC DRAWER / TRAY / BIG SLIDER: |m| ~ child span along
the axis.
- PRISMATIC BUTTON / SMALL TOGGLE: |m| ~ HALF the axial extent
  (buttons press in about half their depth).

When in doubt, prefer the child span along the axis direction
provided
in the metadata.
</magnitude_guidance>

<output_format>
Return STRICTLY valid JSON - a list with one entry per non-fixed
joint:
[
  {
    "part_id": int,
    "joint_type": "revolute|prismatic|continuous|screw",
    "target_range": float,
    "rationale": "1-2 sentences: (i) which sign rule and why, (ii
) which magnitude class and its numeric source."
  }
]

Include ALL non-fixed parts. Do NOT include fixed parts.
</output_format>

<final_instructions>
Think step-by-step for each joint:
1. Identify the semantic class from the part name and initial
frame.
2. Read the yellow arrow's direction from the hint image using
the
   object-frame-to-image mapping above.
3. Apply the right-hand rule (rotation) or arrow-direction rule
   (translation) to choose the SIGN.
4. Pick magnitude from the guidance.
5. Write the rationale.
6. Return only the JSON list.
</final_instructions>
```

I.2 Benchmark VLM Judge Prompts

The benchmark judge uses the prompt components below. At runtime, the evaluator sends the shared system prompt followed immediately by one aspect-specific rubric, prepends three aspect-specific few-shot examples, and then sends the final query with the same image layout as the few-shot examples.

Prompt 8. VLM judge shared system prompt

```
<role>
You are an expert evaluator of computer-generated 3D articulated
objects.
You assess how faithfully a generated model reproduces the
mechanical structure visible in a reference photograph.
</role>

<knowledge_base>
**Joint taxonomy** (the five mechanical joint types):
- **Revolute**: rotation with angular limits (door hinges, lid
pivots, levers).
- **Prismatic**: linear translation (drawers sliding out, push
buttons).
- **Continuous**: unlimited rotation (knobs, wheels, turntable
swivels).
- **Screw**: coupled rotation + translation (threaded bottle caps
, office-chair seat height adjustment, pump nozzles).
- **Fixed**: no motion - used only for the root/base part or
static decorative sub-parts that were incorrectly split during
segmentation.

**Axis placement**:
- Use the reference photograph and your physical intuition to
infer where each joint's axis of rotation or translation should
be.
- Revolute joints: the axis should be at the **pivot edge** - the
 edge where the part is attached to the body. For doors with
visible handles, the hinge is typically on the opposite side; but
 many revolute parts (lids, levers, flaps) have no handle - infer
 the pivot from the part's shape and attachment.
- Prismatic joints: the axis should align with the **sliding
direction** (e.g., along the rail for drawers, perpendicular to
the panel for buttons).
- Continuous joints: the axis passes through the **center of
rotation** (e.g., center of a wheel or knob).
```

Prompt 8. VLM judge shared system prompt (continued)

```
- Screw joints: treated the same as continuous/prismatic for axis
 placement - the axis passes through the rotation center AND
aligns with the translation direction (e.g., a bottle cap's screw
 axis runs along the threaded spindle).

**Part completeness**:
- A drawer part should include the **container slot** (the box
behind the panel). If the container is missing, the drawer is
incomplete.
- A door part should be a **flat panel** only. If it has an
incorrectly attached container box, that is a decomposition error.


**Small controls - leniency vs expectation**:
- **Small flat buttons and printed labels** on a control panel
are surface textures. The model is NOT required to reproduce them
 as separate parts - but if it does generate them, that is fine (
do NOT penalize extra buttons).
- **Prominent knobs/dials** that visibly protrude from the
surface in the reference (e.g., a large washing-machine program
selector, an oven temperature dial) ARE expected as independent
articulable parts. Missing a clearly prominent knob is a
completeness error.
- Rule of thumb: if the control is **flush/flat** on the panel,
it is optional; if it **protrudes in 3D**, it is expected.

**Assembly quality**:
- Parts should fit together tightly, as in a real manufactured
object. Large visible gaps between parts that should be touching
(e.g., a door that floats away from the body, or panels that don'
t align) indicate poor part geometry or placement.

**Handle rules**:
- Handles that are **rigidly fixed** to a door or drawer are NOT
separate articulated parts - they belong to the same link as the
panel they are attached to.
- Only count a handle as a separate part if it has its own
independent motion (e.g., a rotating latch).

**Symmetric rotation** (do NOT penalize):
- A geometrically symmetric part (e.g., a perfectly round wheel
or knob with uniform spokes) rotating around its symmetry axis
will show **no visible change** between frames. This is correct
physics - the rotation is real but visually undetectable. Do NOT
treat this as "no motion." Conversely, if a supposedly symmetric
wheel shows visible wobble or offset during rotation, that
indicates an off-center axis (penalize).
</knowledge_base>

<output_format>
You MUST respond with ONLY a valid JSON object - no other text
before or after.
The object has exactly two fields in this order: `score` (integer
 1-5) first, then `analysis` (a plain-text explanation). Always
open the analysis with "The reference image shows ..." describing
 the real object, then compare the generated model against the
reference and apply the scoring rubric.
Example:
```json
{"score": 4, "analysis": "The reference image shows a cabinet
with two doors and a drawer. The generated model captures all
three parts but ..."}
```
Replace the values with your actual score and analysis.
</output_format>
```

Prompt 9. VLM judge rubric for PART_DECOMP

```
<scoring_rubric aspect="PART_DECOMP">
You will be asked to evaluate the **PART_DECOMP** dimension -
Part Decomposition Quality.

**Scoring rubric:**
FIRST CHECK: If the generated model is an entirely different type
 of object from the reference photograph (e.g., a chair when the
reference shows a washing machine), score **1** immediately - do
not evaluate further.

Scoring procedure:
1. Identify all distinct **functional regions** in the reference
(e.g., a washing machine has door / drum / knob / buttons = 4
regions).
2. For the decomposition of parts that ARE present, start from 5:
   - Parts at correct mechanical boundaries, tight assembly: 5.
   - One minor boundary error or small gap: 4.
   - 1-2 notable errors (wrong merge/split, floating parts): 3.
   - Multiple wrong boundaries: 2.
3. Apply a **missing-part penalty**: deduct 1 point if any
functional region from the reference is missing (merged into body
 or absent). Deduct at most 1 regardless of how many are missing
- this dimension focuses on decomposition quality, not
completeness.
4. Functionally reasonable extra parts: no penalty. Unreasonable
splits: -1.
Minimum score is 1.

**Analysis guide:**
Compare the generated model's part structure to the reference.
IMPORTANT: Meshes with the **same color** that are **adjacent or
touching** belong to the **same rigid link** - they are NOT
separate parts. However, two distant regions that happen to share
 a similar color may be different parts (the palette has limited
colors). Use spatial adjacency, not color alone, to decide part
identity.
Check: (1) Are parts separated at the right mechanical boundaries
? (2) Are fixed handles correctly merged with their panel? (3) Do
 parts fit together tightly, or are there visible gaps? (4) Are
any parts from the reference missing or incorrectly merged?
</scoring_rubric>
```

Prompt 10. VLM judge rubric for PART_COMPLETE

```
<scoring_rubric aspect="PART_COMPLETE">
You will be asked to evaluate the **PART_COMPLETE** dimension -
Part Completeness.

**Scoring rubric:**
FIRST CHECK: If the generated model is an entirely different type
 of object from the reference photograph (e.g., a chair when the
reference shows a washing machine), score **1** immediately - do
not evaluate further.

Scoring procedure:
1. Identify all distinct **functional regions** in the reference
(e.g., a washing machine: door / drum / knob / buttons = 4
regions; a microwave: door / knob = 2 regions).
2. Count how many are present in the generated model. Score by
the fraction present:
   - All present: 5.
   - One missing: 4.
   - About half missing: 3.
```

Prompt 10. VLM judge rubric for PART_COMPLETE (continued)

```
   - Most missing: 2.
   - Nearly all missing: 1.
3. Drawers must include the container slot; doors must be flat
panels.
4. Functionally reasonable extra parts: no penalty.

**Analysis guide:**
Count the functional regions in the reference image (doors,
drawers, lids, knobs, drums, etc.). Check if each has a
corresponding part in the model. Score based on the fraction
present.
</scoring_rubric>
```

Prompt 11. VLM judge rubric for MOTION

```
<scoring_rubric aspect="MOTION">
You will be asked to evaluate the **MOTION** dimension -
Articulation Motion Quality.

**Scoring rubric:**
FIRST CHECK: If the generated model is an entirely different type
 of object from the reference photograph (e.g., a chair when the
reference shows a washing machine), score **1** immediately - do
not evaluate further.

Scoring procedure:
1. From the reference, identify expected motions and assign a **
point budget** (total = 4). Distribute points across functional
parts roughly by importance (e.g., a microwave with door + knob
-> door=2.5, knob=1.5; a cabinet with door + drawer -> door=2,
drawer=2).

2. For each expected part, compare rest state vs fully-open state.
 Award its full points if the motion is **accurate**: correct
axis placement (e.g., hinge at the correct edge), correct
direction, plausible range (e.g., ~90 degrees for a microwave
door), and no interpenetration.

3. Deduct from that part's budget:
   - **Minor issue** (slight clipping at extremes, range slightly
 off, small drift from seam): deduct ~half of that part's points.
   - **Major issue** (wrong direction + penetration, severe skew,
 translating instead of rotating, or completely wrong range):
deduct all of that part's points.
   - **Part entirely missing** (expected from reference but
absent or static in the sequence): deduct all of that part's
points.

4. **Extra parts** not expected from the reference: no penalty if
 their motion is reasonable and physically plausible. If their
motion is broken or they are functionally unrelated to the object
, deduct 1 point.

Note: A geometrically symmetric part (round wheel, uniform knob)
rotating around its symmetry axis will look identical across
frames - correct physics, NOT a lack of motion.

Floor: minimum **2** if the model is the correct type of object
and has at least one correctly moving part. Score **1** only for
completely nonsensical motion or an entirely different object.

**Analysis guide:**
```

Prompt 11. VLM judge rubric for MOTION (continued)

```
Compare the rest state (Frame 1) with the open states, especially
 the fully-open state (last frame). For each moving part:
1. Is the motion type correct (rotation vs translation)?
2. Is the axis/pivot at the right location (hinge at edge, not
center)?
3. Is the direction correct (outward, not inward)?
4. Is the range plausible (doors ~90-120 degrees, drawers extend
their depth)?
5. Is there any interpenetration? Check BOTH camera views.
Also check: are there expected motions from the reference that
are missing?
</scoring_rubric>
```

Prompt 12. VLM judge rubric for JOINT_TYPE

```
<scoring_rubric aspect="JOINT_TYPE">
You will be asked to evaluate the **JOINT_TYPE** dimension -
Joint Type Correctness.

**Scoring rubric:**
FIRST CHECK: If the generated model is an entirely different type
 of object from the reference photograph (e.g., a chair when the
reference shows a washing machine), score **1** immediately - do
not evaluate further.

The model has {total_joints} actuated joint(s) in total ({
type_summary}). The top {n_joints} by importance are shown above.

Score in two steps:

**Step 1 - Type correctness** of the SHOWN joints (fraction
correct):
- All correct: 5.
- One wrong: 4.
- About half wrong: 3.
- Most wrong: 2.

**Step 2 - Completeness**: compare the model's total joint count
to the number of functional regions expected from the reference.
If the model has significantly fewer joints than expected (e.g.,
ref has 4 drawers but model has only 1), reduce proportionally (
even if the shown joints are correct).

Floor: if the model resembles the reference and has at least some
 correct joints, minimum score is **2**. Score 1 is reserved for
all-wrong types or an entirely different object.

**Analysis guide:**
Step 1: Look at the reference photograph. Identify the expected
articulable parts and their types (doors=revolute, drawers=
prismatic, knobs=continuous, etc.).
Step 2: You are shown {n_joints} joint-highlight image(s) out of
the model's {total_joints} total joints ({type_summary}). For
each shown joint, check if its claimed type matches the expected
type from the reference.
Step 3: Compare the model's total joint count to what the
reference suggests. If many expected parts are missing, reduce
the score proportionally.
</scoring_rubric>
```

Prompt 13. VLM judge rubric for JOINT_AXIS

```
<scoring_rubric aspect="JOINT_AXIS">
You will be asked to evaluate the **JOINT_AXIS** dimension -
Joint Axis Placement and Orientation.

**Scoring rubric:**
FIRST CHECK: If the generated model is an entirely different type
 of object from the reference photograph (e.g., a chair when the
reference shows a washing machine), score **1** immediately - do
not evaluate further.

The model has {total_joints} actuated joint(s) in total ({
type_summary}). The top {n_joints} by importance are shown above.

Scoring procedure:

**Step 1 - Build a point budget (total = 4, plus 1 base point).**
Distribute the 4 points across the shown joints by importance in
the reference object. For example:
- Single dominant joint (e.g., a microwave door): 4 points ->
direction=2, position=2.
- Two equally important joints (e.g., cabinet door + drawer): 2
points each -> direction=1, position=1 per joint.
- Four joints (e.g., 2 oven doors + 2 racks): 1 point each ->
direction=0.5, position=0.5 per joint.
- If some joints are clearly more important than others, give
them a larger share (e.g., main door = 2.5, secondary buttons =
0.5 each).

**Step 2 - Per-joint deductions.** For each shown joint, compare
the predicted green axis against the expected axis and deduct
from that joint's allocated points:
  (a) **Direction / orientation** (half of the joint's points):
      - Correct direction: no deduction.
      - Slightly off (a few degrees of skew): lose half of the
direction sub-budget.
      - Completely wrong axis (e.g., vertical instead of
horizontal, or perpendicular to correct): lose the full direction
 sub-budget.
  (b) **Origin / position** (the other half):
      - Exactly on the expected mechanical location: no deduction.

      - Slightly offset (axis floats near but not on the seam, a
small shift from the knob center): lose half of the position sub-
budget.
      - Clearly on the wrong side of the part, or far away from
the correct location: lose the full position sub-budget.

Inspect the zoomed-in inset (if present) and BOTH camera views -
an axis may look right from one angle but float from another.

**Step 3 - Completeness**: compare the model's total joint count
to the number of functional regions expected from the reference.
If the model has significantly fewer joints than expected (e.g.,
ref has 4 drawers but the model has only 1), reduce
proportionally (even if the shown joint axes are all correct).

Sum the remaining points and add the 1 base point to obtain the
final 1-5 score (round to the nearest integer).

Floor: minimum **2** if the model resembles the reference and has
 at least some correct axes. Score **1** only for random axes or
an entirely different object.

**Analysis guide:**
```

Prompt 13. VLM judge rubric for JOINT_AXIS (continued)

```
Follow the rubric's point-budget procedure.  First inspect the
reference to identify how many joints to expect and how to
distribute the 4-point budget across them.  Then, for each shown
joint, examine the green axis line in BOTH camera views and the
zoom inset (if present), applying the direction and position
deductions:
- **Revolute**: direction = parallel to the hinge axis; position
= on the pivot edge/seam (not floating, not at center).
- **Prismatic**: direction = along the sliding axis; position =
anywhere on that line is acceptable.
- **Continuous**: direction = symmetry axis; position = through
the rotation center.
- **Screw**: direction = along the threaded spindle (the combined
 rotation + translation axis); position = through the spindle
center (same criteria as continuous).
A common failure mode is correct direction but offset position -
the axis floats parallel to where it should be.
</scoring_rubric>
```

References

Mahmoud Ahmed, Xiang Li, Arpit Prajapati, and Mohamed Elhoseiny. 2024. 3dcompat200: Language grounded large-scale 3d vision dataset for compositional recognition. *Advances in Neural Information Processing Systems* 37, 135770–135782.

Ziang Cao, Zhaoxi Chen, Liang Pan, and Ziwei Liu. 2025a. PhysX-3D: Physical-Grounded 3D Asset Generation. In *Advances in Neural Information Processing Systems*, Vol. 38. 93771–93784.

Ziang Cao, Fangzhou Hong, Zhaoxi Chen, Liang Pan, and Ziwei Liu. 2025b. PhysX-Anything: Simulation-Ready Physical 3D Assets from Single Image. *arXiv preprint arXiv:2511.13648* (2025).

Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, et al. 2025. Sam 3: Segment anything with concepts. *arXiv preprint arXiv:2511.16719* (2025).

Angel X Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, et al. 2015. ShapeNet: An Information-Rich 3D Model Repository. *arXiv preprint arXiv:1512.03012* (2015).

Zoey Chen, Aaron Walsman, Marius Memmel, Kaichun Mo, Alex Fang, Karthikeya Vemuri, Alan Wu, Dieter Fox, and Abhishek Gupta. 2024. Urdformer: A pipeline for constructing articulated simulation environments from real-world images. *arXiv preprint arXiv:2405.11656* (2024).

Matt Deitke, Ruoshi Liu, Matthew Wallingford, Huong Ngo, Oscar Michel, Aditya Kusupati, Alan Fan, Christian Laforte, Vikram Voleti, Samir Yitzhak Gadre, Eli VanderBilt, Aniruddha Kembhavi, Carl Vondrick, Georgia Gkioxari, Kiana Ehsani, Ludwig Schmidt, and Ali Farhadi. 2023. Objaverse-XL: A Universe of 10M+ 3D Objects. In *Advances in Neural Information Processing Systems*, Vol. 36. 35799–35813.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems* 36 (2023), 10088–10115.

Lihe Ding, Shaocong Dong, Yaokun Li, Chenjian Gao, Xiao Chen, Rui Han, Yihao Kuang, Hong Zhang, Bo Huang, Zhanpeng Huang, et al. 2025. FullPart: Generating each 3D part at full resolution. *arXiv preprint arXiv:2510.26140* (2025).

Huan Fu, Rongfei Jia, Lin Gao, Mingming Gong, Binqiang Zhao, Steve Maybank, and Dacheng Tao. 2021. 3D-FUTURE: 3D Furniture Shape with Texture. *International Journal of Computer Vision* 129, 12 (2021), 3313–3337.

Google DeepMind. 2025a. Introducing Nano Banana Pro (Gemini 3 Pro Image). Google Blog. https://blog.google/innovation-and-ai/products/nano-banana-pro/ Accessed: 2026-05-08.

Google DeepMind. 2025b. Introducing Veo 3.1 and new creative capabilities in the Gemini API. Google Developers Blog. https://developers.googleblog.com/introducing-veo-3-1-and-new-creative-capabilities-in-the-gemini-api/ Accessed: 2026-05-08.

Denys Iliash, Hanxiao Jiang, Yiming Zhang, Manolis Savva, and Angel X Chang. 2026a. S2o: Static to openable enhancement for articulated 3d objects. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. 6785–6795.

Denys Iliash, Jiayi Liu, Egor Fokin, Qirui Wu, Ali Mahdavi Amiri, Manolis Savva, and Angel X. Chang. 2026b. Artiverse: A Diverse and Physically Grounded Dataset for Articulated Objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 8932–8942.

Hanxiao Jiang, Yongsen Mao, Manolis Savva, and Angel X Chang. 2022. OPD: Single-View 3D Openable Part Detection. In *European Conference on Computer Vision*. Springer, 410–426.

Mukul Khanna, Yongsen Mao, Hanxiao Jiang, Sanjay Haresh, Brennan Shacklett, Dhruv Batra, Alexander Clegg, Eric Undersander, Angel X Chang, and Manolis Savva. 2024. Habitat Synthetic Scenes Dataset (HSSD-200): An Analysis of 3D Scene Scale and Realism Tradeoffs for ObjectGoal Navigation. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 16384–16393.

Sebastian Koch, Albert Matveev, Zhongshi Jiang, Francis Williams, Alexey Artemov, Evgeny Burnaev, Marc Alexa, Denis Zorin, and Daniele Panozzo. 2019. ABC: A Big CAD Model Dataset for Geometric Deep Learning. In *2019 IEEE/CVF conference on computer vision and pattern Recognition (CVPR)*. IEEE, 9593–9603.

Eric Kolve, Roozbeh Mottaghi, Winson Han, Eli VanderBilt, Luca Weihs, Alvaro Herrasti, Matt Deitke, Kiana Ehsani, Daniel Gordon, Yuke Zhu, Aniruddha Kembhavi, Abhinav Gupta, and Ali Farhadi. 2017. AI2-THOR: An Interactive 3D Environment for Visual AI. *arXiv preprint arXiv:1712.05474* (2017).

Long Le, Jason Xie, William Liang, Hung-Ju Wang, Yue Yang, Yecheng Jason Ma, Kyle Vedder, Arjun Krishna, Dinesh Jayaraman, and Eric Eaton. 2025. Articulate-anything: Automatic modeling of articulated objects via a vision-language foundation model. In *International Conference on Learning Representations*, Vol. 2025. 17578–17602.

Ruining Li, Yuxin Yao, Chuanxia Zheng, Christian Rupprecht, Joan Lasenby, Shangzhe Wu, and Andrea Vedaldi. 2025. Particulate: Feed-Forward 3D Object Articulation. *arXiv preprint arXiv:2512.11798* (2025).

Zhe Li, Xiang Bai, Jieyu Zhang, Zhuangzhe Wu, Che Xu, Ying Li, Chengkai Hou, and Shanghang Zhang. 2026. URDF-Anything: Constructing Articulated Objects with 3D Multimodal Language Model. *Advances in Neural Information Processing Systems* 38 (2026), 94974–95002.

Haotong Lin, Sili Chen, Junhao Liew, Donny Y Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. 2025. Depth anything 3: Recovering the visual space from any views. *arXiv preprint arXiv:2511.10647* (2025).

Jiayi Liu, Denys Iliash, Angel Chang, Manolis Savva, and Ali Mahdavi Amiri. 2025a. Singapo: Single image controlled generation of articulated parts in objects. In *International Conference on Learning Representations*, Vol. 2025. 97511–97532.

Minghua Liu, Mikaela Angelina Uy, Donglai Xiang, Hao Su, Sanja Fidler, Nicholas Sharp, and Jun Gao. 2025b. Partfield: Learning 3d feature fields for part segmentation and beyond. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 9704–9715.

Qingming Liu, Xinyue Yao, Shuyuan Zhang, Yueci Deng, Guiliang Liu, Zhen Liu, and Kui Jia. 2026. Pact: Part-decomposed single-view articulated object generation. *arXiv preprint arXiv:2602.14965* (2026).

Xinhai Liu, Zhizhong Han, Yu-Shen Liu, and Matthias Zwicker. 2021. Fine-Grained 3D Shape Classification with Hierarchical Part-View Attention. *IEEE Transactions on Image Processing* 30 (2021), 1744–1758.

Zhao Mandi, Yijia Weng, Dominik Bauer, and Shuran Song. 2024. Real2code: Reconstruct articulated objects via code generation. *arXiv preprint arXiv:2406.08474* (2024).

Nikhila Ravi, Jeremy Reizenstein, David Novotny, Taylor Gordon, Wan-Yen Lo, Justin Johnson, and Georgia Gkioxari. 2020. Accelerating 3d deep learning with pytorch3d. *arXiv preprint arXiv:2007.08501* (2020).

Hamid Rezatofighi, Nathan Tsoi, JunYoung Gwak, Amir Sadeghian, Ian Reid, and Silvio Savarese. 2019. Generalized intersection over union: A metric and a loss for bounding box regression. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 658–666.

Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).

Hanqing Wang, Jiahe Chen, Wensi Huang, Qingwei Ben, Tai Wang, Boyu Mi, Tao Huang, Siheng Zhao, Yilun Chen, Sizhe Yang, et al. 2024. Grutopia: Dream general robots in a city at scale. *arXiv preprint arXiv:2407.10943* (2024).

Hengyi Wang, Haizhou Shi, Shiwei Tan, Weiyi Qin, Wenyuan Wang, Tunyu Zhang, Akshay Nambi, Tanuja Ganu, and Hao Wang. 2025b. Multimodal needle in a haystack: Benchmarking long-context capability of multimodal large language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 3221–3241.

Penghao Wang, Yiyang He, Xin Lv, Yukai Zhou, Lan Xu, Jingyi Yu, and Jiayuan Gu. 2025a. Partnext: A next-generation dataset for fine-grained and hierarchical 3d part understanding. *arXiv preprint arXiv:2510.20155* (2025).

Karl DD Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G Lambourne, Armando Solar-Lezama, and Wojciech Matusik. 2021. Fusion 360 Gallery: A Dataset and Environment for Programmatic CAD Construction from Human Design Sequences. *ACM Transactions on Graphics (TOG)* 40, 4 (2021), 1–24.

Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 2015. 3D ShapeNets: A Deep Representation for Volumetric Shapes. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 1912–1920.

Fanbo Xiang, Yuzhe Qin, Kaichun Mo, Yikuan Xia, Hao Zhu, Fangchen Liu, Minghua Liu, Hanxiao Jiang, Yifu Yuan, He Wang, Li Yi, Angel X. Chang, Leonidas Guibas, and Hao Su. 2020. SAPIEN: A SimulAted Part-Based Interactive ENvironment. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*.